Work avoidance for efficient sparse set algebra

Marcus D. R. Klarqvist[1]

[1]**[affiliation]**

ABSTRACT

Computing the cardinality of a set intersection, $|A \cap B|$, over a shared universe of m elements underlies similarity search, inverted-index query evaluation, and graph analytics. The standard approach stores each set as a dense bitmap and evaluates `popcount(A AND B)` with the widest available vector instruction, costing $\Theta(m)$ regardless of either set's cardinality. On real, skewed data most pairs have a near-empty side, so most of that fixed cost is spent ANDing zeros against zeros. Compressed bitmaps adapt to this only per fixed-size chunk, dispatching among array, bitset, and run containers against a threshold fixed at compile time. Here we show that treating the representation pairing itself as the object of choice — selected per pair from measured rather than modelled properties, hoisted above the innermost loop — recovers most of that fixed cost. A throughput-optimized kernel wins none of **[count of asymmetric measurement points tested]** asymmetric measurement points tested; best-cell selection beats a tuned reference implementation on every one of **[count of real corpora tested]** real corpora, at a selection cost of **[selection cost as percent of runtime under the hoisted selection policy]** of runtime. We also delimit where it does not pay: a mid-density band in which the plain bitmap kernel remains the right choice. Avoiding work, not accelerating it, wins once row densities within one universe are heterogeneous.

Computing the cardinality of a set intersection, $|A \cap B|$, over a shared universe of possible elements is a core primitive in similarity search and near-duplicate detection[1,2], in chemical fingerprint screening[3–5], in graph co-occurrence analysis, and in the boolean query evaluation underneath inverted and bitmap indexes. It is a single-operation primitive: whatever the surrounding workload, what a system pays is the cost of one operation between two sets, and that cost is fixed by how each of the two is represented before the operation is ever issued.

The default way to compute $|A \cap B|$ represents each set as a dense bitmap over a shared universe of m possible elements and evaluates `popcount(A AND B)` with the widest available vector instruction. This costs $\Theta(m)$ irrespective of the cardinality of either set or of their intersection — every one of the m bits is read and combined regardless of how many are set — and the underlying AND-then-popcount kernel is itself a solved problem, engineered close to the throughput limit of the hardware[6]. For a single pair, or for sets that occupy a substantial fraction of the universe, this is exactly the right way to compute the answer.

Real degree and frequency distributions are rarely uniform: a small number of categories carry most of the mass while a long tail is rare. The canonical form of this skew — the i-th most common category scaling as θ/i — was derived for population-genetic allele-frequency spectra[7] and recurs, in shape if not in constant, in item popularity, node degree, and category-membership counts across many other kinds of data. Under such skew, row density spans orders of magnitude within a single corpus, so most rows are sparse and most pairs have at least one near-empty side — yet the $\Theta(m)$ bitmap kernel pays the same fixed cost regardless, spending most of that cost combining zeros against zeros. That waste is not incidental to one kind of data: every alternative encoding of the same set — a sorted array, a list of runs, a fill-compressed word stream, or any of these applied to the complement — is exact and interconvertible with the bitmap, sized by the set rather than by the universe, and smaller than the bitmap by a factor that grows without bound as either operand moves away from density one half; only near density one half does no such encoding help (Supplementary Note 1).

Roaring is the state of the art for exactly this problem, and it already dispatches, per fixed-width chunk, among an array, a bitset, and a run container[8,9] — the same coarse-grained skip toward a cheaper structure that inverted-index systems such as BitFunnel apply to disjointness testing[10]. It is excellent, it is deployed at scale, and this paper does not attempt to out-engineer it at its own data structure. Its adaptivity is nonetheless fixed in granularity and static in time: which container a chunk receives is decided by a compile-time cardinality threshold applied to that chunk alone, chosen once at ingest for a workload built around a single pairwise operation (Methods, "Container census"). That same fixed commitment prices a run container against a bitset by iterating every run and charging for its length — a cost a length-independent index would remove. A caller willing to narrow the question further, to results above a similarity threshold or to operands within a size band, can be given even more (Supplementary Note 2).

Two questions follow directly. Given that popcount is a solved problem and Roaring already dispatches on container type per chunk, is there anything left for representation choice to buy? And if a wider representation matrix does buy something, can the choice itself be made cheaply enough — for one operation between two large sets, and for a batch of many small ones —

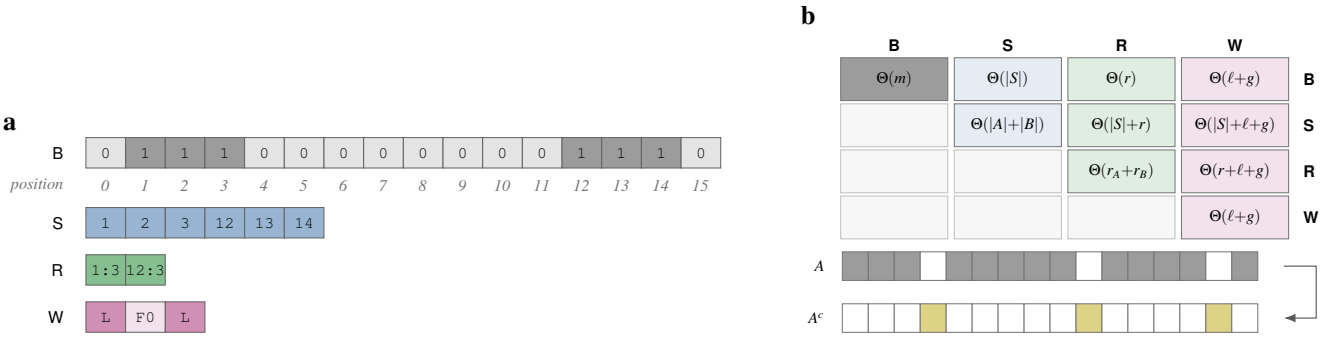

Figure 1. The object of study: equivalent encodings of one set, and the matrix of kernels that pairing them gives. a, One set of six elements over a 16-position universe, encoded four ways. One cell is one stored item, so a strip's length is the number of items that encoding must touch: 16 bits for B ($\Theta(m)$, whatever the set holds), six values for S ($\Theta(|X|)$), two (*start*:*length*) records for R ($\Theta(r)$, whatever the runs' lengths), and three words — two literal and one fill — for W ($\Theta(\ell+g)$). Note that a cell is a bit in B and a word in the others, so the strips compare work, not bytes. Strong fill marks set positions, and the position ruler applies to B alone, the only positional encoding. **b**, Pairing two representations gives a kernel with its own cost function. The matrix is symmetric, so ten cells are distinct, and *only* B×B is linear in the universe m; every other cell is sub-linear in it. Above density one half a side is handled through its complement (C), shown beneath: a dense operand becomes a sparse one and re-enters the same ten cells, with the answer recovered by inclusion–exclusion. Costs are derived in Methods. Schematic; no measured data.

that the saving survives the deciding?

Here we treat the representation pairing itself as the object of choice. We define a pairing matrix of **[count of representation-pairing cells evaluated]** cells over dense-bitmap, sorted-array, run, and fill-compressed representations and their complements, and we measure, per cell, which pairing is fastest across the full density range, on **[count of microarchitectures tested]** microarchitectures, and on **[count of real corpora tested]** real corpora drawn from graph, text, and tabular sources. Cost across that range is U-shaped rather than monotonic, so the operative variable is distance from density one half and not sparsity. A throughput-optimized kernel wins none of **[count of asymmetric measurement points tested]** asymmetric measurement points tested; the decision itself stays within a small, budgeted fraction of runtime once it is hoisted above the innermost loop; the winning pairing migrates with density across every real corpus tested; and the reference implementation's own container statistics show why a threshold fixed at compile time misprices a corpus that is dense in aggregate. We also report where the approach does not pay: a mid-density band in which the plain bitmap kernel is, and remains, the right choice.

Results

We evaluated representation-pairing kernels for set-intersection cardinality. Throughout, we use *cell* to denote one representation pairing (for example B×S), *variant* to denote one implementation of a cell, and *pairing matrix* to denote the full set of cells; unless stated otherwise, every ratio is reported against a `run_optimize`-tuned CRoaring baseline. Figure 1 lays out that matrix: the five representation families, the driving variable each pairing's cost is proportional to, and the path from precomputed row metadata through the selector to a chosen kernel.

The mechanisms reported here form a progression ordered by what a caller gives up, not by what each costs. Choosing a representation (Fig. 1a–b) and consulting a zone map (Fig. 2a) give up nothing: the question and the answer are unchanged, and only the work changes. Prefix filtering and a size bound narrow the question instead — only pairs above a similarity threshold, or within a size band — and are available only because the caller said what they wanted; they are opt-in, never substituted for the first two, and their return is analytic rather than measured here, reaching as little as a few per cent of pairs surviving at a high similarity threshold (Methods, "What makes work avoidable"; see Supplementary Note 3 and Supplementary Fig. S1 for the derivation and the composition rule that governs when the two may be multiplied together). Batch amortisation leaves the contract unchanged again but needs many operations rather than one, because what it removes is repetition rather than work.

The cost of intersection is U-shaped in density

We separate the cost of intersection along density – more precisely, each row's distance from density one half. To determine whether the fixed cost assumed of bitmap–bitmap intersection actually holds across the range it is assumed to hold across, and where the remaining cells beat it, we swept density from a single set bit to a fully set universe and measured every one of **[count of pairing cells swept in the density sweep]** cells at each point. Both sides were swept at matched density with the

universe pinned at **[universe size held fixed across the density sweep, in bits]** bits, so that cache residency – **[cache level the density sweep is resident in, e.g. L2]** throughout – stayed constant along the axis instead of confounding it. The sweep in Fig. 2 covers the uniform spectrum; the $1/i$ spectrum, exercised identically, is reported in Supplementary Fig. **[number of the supplementary figure carrying the 1/i density sweep]**.

Bitmap–bitmap cost was flat across the swept range – **[B×B cost, ns per pair, range across the density sweep]** at the cache residency stated above, spanning **[order-of-magnitude span of the density sweep]** orders of magnitude of density – which turns the fixed-cost premise underlying the standard approach from an argument into a measurement. Against that flat line, the best cell at the sparsest density point beat B×B by **[best-cell speedup over B×B at the sparsest density point]**, and the advantage grew with universe size.

The two curves crossed at approximately **[crossover density, order of magnitude, stated as an approximate value]** – an approximate value, since the exact crossover moves under re-measurement even though its existence does not. Near density one, the curve turned again: a row that is almost entirely set has a complement that is almost entirely empty, and we expect a complement-based cell, C×B or C×C, to win the dense tail as a designed strategy, beating the margin achieved when R×R wins only incidentally, beating B×B by **[dense-tail win ratio over B×B at the densest measured point]**. The result is a U, not a monotonic curve, and it corrects the framing this project began with: the operative variable is not sparsity but distance from density one half, since a row that is 99.99% set is exactly as compressible as one that is 0.01% set (Fig. 2).

Each mechanism above is a bet that the data deviates from randomness, and the size of the win at the sparse extreme, the location of the crossover, and the two-sided condition under which a zone map is worth consulting at all follow from a closed-form counting argument against the least favourable (independent-placement) arrangement of elements, given in full in Supplementary Note 4. Inside the band where nothing beats B×B (Fig. 2d), no encoding is smaller than the bitmap and no bin is empty, so the return there comes from throughput rather than avoidance — the axis the vectorized `AND`-and-popcount literature has already carried a long way[6].

Avoiding work beats accelerating it, and the gap widens with working-set size

Where the subsection above varied density across a fixed set of strategies, this one holds corpus shape fixed and instead separates *which kind of speedup* wins – accelerating work or avoiding it – across working-set size and microarchitecture. Having established that most cells beat B×B above, we asked what mechanism is responsible for the asymmetric cells' advantage, since the answer determines what those kernels should be built around. We raced all **[count of asymmetric pairing cells raced]** asymmetric cells (B×S, B×R, B×W, S×R, S×W) at **[count of corpus shapes tested]** corpus shapes – **[count of asymmetric-cell measurement points, expected twenty-five]** measurement points in total – with work-avoidance strategies (a rank lookup, a zone-map skip[10], a galloping search) raced head to head against SIMD-throughput variants of the same cells. Separately, we repeated the comparison as corpus footprint crossed from L1- to L2- to DRAM-resident, on **[count of microarchitectures in the working-set sweep]** microarchitectures.

The winning variant at essentially every asymmetric measurement point was a rank lookup, a zone-map skip, or a search-strategy choice, never a wider vector: **[count of asymmetric measurement points won by a work-avoidance strategy]** of **[total asymmetric measurement points, expected twenty-five]** (Fig. 3a). Read as a map rather than a count, that panel gives the winning strategy class for every cell at every corpus shape, and the work-avoidance class occupies all of it except the B×B column. This is the positive result the asymmetric cells are built around: each is already proportional to the small side of the pair, so the available saving lies in not touching the large side at all, and the measurement confirms that is exactly where the winning variants put their effort.

This outcome is scoped to the *asymmetric* cells alone, and the scope matters because the obvious misreading is available and wrong. Those cells are already work-proportional to the smaller operand, so the quantity a wider vector could accelerate is small by construction; there is little there for throughput to win, and the remaining saving lies in not touching the larger operand. The complement holds with equal force and is measured here too. B×B is the one cell whose work is, in the generic case, not avoidable — it is the red waist of Fig. 2d — and it is exactly there that throughput optimization pays: adding an AVX-512 `VPOPCNTQ` path took that cell from **[scalar B×B cost, ns per pair]** to **[AVX-512 B×B cost, ns per pair]**, an improvement of **[B×B speedup from the AVX-512 `VPOPCNTQ` path]**, and it was the right investment for precisely the reason the asymmetric cells did not need one. The finding is not that vectorization fails to help; it is that the two mechanisms have disjoint domains, and that the density analysis of the preceding subsection tells a system which domain it is in before it commits.

The same asymmetry widened with working-set size. On every microarchitecture measured, the SIMD-throughput advantage on B×B – the one cell whose underlying work is, in the generic case, not avoidable – shrank as the corpus grew past L2 residency, from **[SIMD speedup over scalar B×B at L2 residency, per host]** at L2 residency to **[SIMD speedup over scalar B×B at DRAM residency, per host]** at DRAM residency, while the work-avoidance advantage on the asymmetric cells held or grew over the same range, from **[work-avoidance speedup at L2 residency, per host]** to **[work-avoidance speedup at DRAM residency, per host]**. We expect this widening to hold as genuine growth, not merely a plateau, on every host measured,

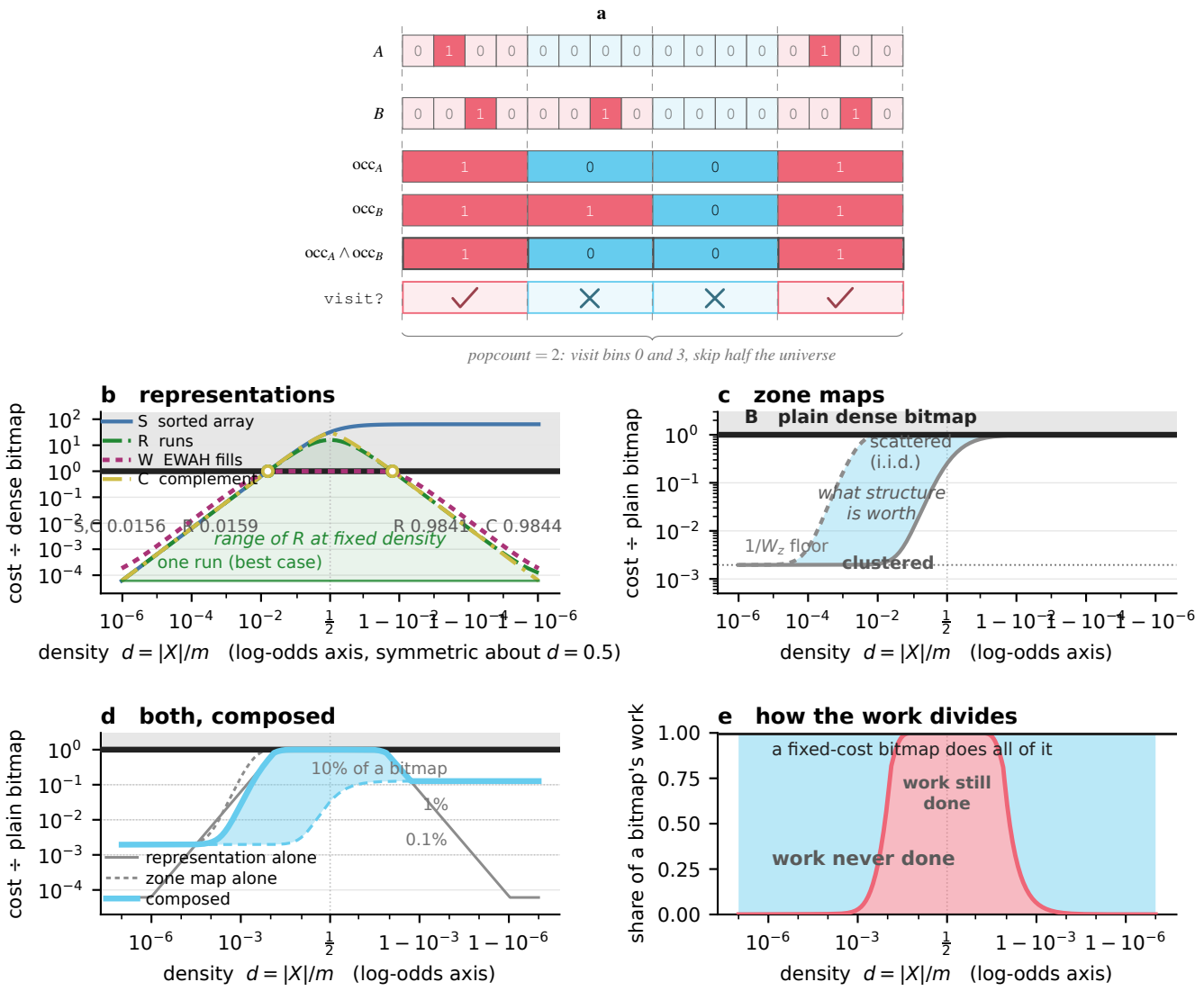

Figure 2. What a fixed cost leaves on the table, analytically. Derived, not measured; every panel models *work* rather than time. Reading key: hue names a representation, so mechanisms are grey and separated by line style; red marks work still done, cyan work avoided, and grey shading the region dearer than a plain bitmap. **a**, A zone map within one pair. Each bin carries one bit recording whether that operand holds anything in it, and $\mathrm{occ}_A \wedge \mathrm{occ}_B$ is the AND of the two summaries. Follow bin 1: B occupies it and A does not, so the pair skips the bin even though one side holds data there. The popcount of the AND is the *exact* number of bins to visit, and a popcount of zero proves the operands disjoint without reading either. Digits and glyphs repeat every verdict, so the panel reads without colour. **b**, Expected cost of each representation for a uniformly random set, as a multiple of the dense-bitmap cost, on a log-odds density axis. Every cost is proportional to the universe m, so this panel is scale-invariant and the crossovers are universal constants: open circles at $d = 1/w = 0.0156$ for S and C, 0.0159 for R, and their complements. The bitmap is the line at 1. Shading marks the range of R at fixed density — one run at best, the i.i.d. value at worst — which is the deviation the method exploits. **c**, The same for a zone map. Arrangement, not density, sets the share of bins an operand occupies: at $d = 0.01$ and $W_z = 512$ the scattered and clustered extremes differ by 482×, so the i.i.d. curve floors what a summary is worth. **d**, The two composed. Filtering concentrates density, so the representation used inside a surviving bin is the one for that bin's local density. Clustered, the composition costs 0.195% of a bitmap across the sparse range at $W_z = 512$, rising to 12.7% only in the dense half; the knee sits at $d = 1/8$ for every bin width. **e**, The same as a division of labour: the share of a bitmap's work never done against the share still done, taking the best of the four options at each density. 99.8% is never done at $d = 10^{-4}$; the waist about $d = \frac{1}{2}$ is where a bitmap is the right kernel. Measurements on real corpora appear on these axes in Supplementary Fig. **[number]**.

FIGURE PLACEHOLDER

(a) winning strategy class for every cell × corpus shape; (b) SIMD and work-avoidance speedup on B×B against corpus footprint from L1 to DRAM residency, by host

Figure 3. Vectorization's advantage decays with working-set size; work avoidance does not. a, Strategy map: for each pairing cell (columns) at each corpus shape (rows), the class of the winning variant — work avoidance (rank lookup, zone-map skip or search-strategy choice) versus SIMD throughput — over **[total asymmetric measurement points]** measured cell–shape points. B×B is the only cell in which a vectorized variant is available to win at all. **b**, Speedup over a common scalar B×B baseline against corpus footprint, on **[host count in the working-set sweep]** hosts spanning **[list of ISAs covered by the working-set sweep]**: a SIMD dense-bitmap variant (solid) and a zone-map-planned variant of the same cell (dashed), so the two series differ only in mechanism. Line style encodes mechanism, colour encodes host. Vertical guides mark the boundary between L2-, last-level-cache- and DRAM-resident footprints on **[reference host for the cache-boundary guides]**; the footprint-to-cache-level mapping differs by host (Methods, "Benchmark protocol and cache-residency statement"). Horizontal dashed line at 1.0× (no benefit over scalar). Corpus shape held fixed at **[corpus shape held fixed across the working-set sweep]** throughout **b**.

resolving the asymmetry between hosts observed previously. The two mechanisms therefore scale in opposite directions with the memory hierarchy: vector width buys relatively less as the hierarchy gets less friendly to it, on the one cell whose work cannot in the generic case be avoided, while the cells built around skipping keep or extend their edge (Fig. 3b).

Selection must be hoisted, and measured rather than modelled

The two subsections above charge only the cost of computing once a cell is chosen; this one charges the cost of the choice itself. The cells characterized above are only a saving if selecting among them is nearly free, so we measured the cost of that selection at three granularities against a runtime budget of **[selection-cost budget, as a fraction of runtime]** of total runtime. We compared selecting per pair – a decision made at every (i, j) – against hoisting the decision to tile granularity, where a whole pre-partitioned block of pairs shares one kernel choice, and against probe-and-commit, the $\varepsilon \to 0$ policy from Micro Adaptivity[11], which times a handful of a tile's pairs under the candidate kernels and commits to the winner for the rest. We report each policy's cost as a percentage of total runtime, and its regret against a bucket oracle that already knows the winning cell, on **[count of hosts the selection-cost sweep was measured on]** hosts. See Methods, "Selection policies: per-pair, per-tile, and probe-and-commit," for the budget, the tile geometry and the probe count.

What those policies have in common is the fourth mechanism of the ladder above: the contract is unchanged and the answer is identical, but the saving comes from paying once for a quantity that would otherwise repeat across every pair of a batch — row metadata, the selection decision itself, or operand data movement. The measurements reported here were taken in that regime, which applies wherever there are enough operations to share a decision; see Supplementary Note 5 and Supplementary Fig. S2 for which quantities repeat and how each is amortised.

Per-pair selection failed that budget outright: **[per-pair selection cost, percent of runtime, range across hosts and corpora]** of runtime, well beyond what the budget allows – the saving established above is real, and at per-pair granularity the decision consumes it. Hoisting the decision passed the same budget with **[headroom of the hoisted policies against the budget, as a factor]** of headroom: per-tile selection cost **[per-tile selection cost, percent of runtime]**, probe-and-commit **[probe-and-commit selection cost, percent of runtime]** (Fig. 4a).

Against the bucket oracle, measured selection beat modelled selection outright: regret ranked **[regret ranking and magnitude, probe-and-commit vs. per-tile vs. all-bitmap-always vs. a closed-form per-pair cost model]**, with the closed-form model alone landing worse than making no selection at all (Fig. 4b). This is the direct justification for probe-and-commit over a cost model: predicting the winning cell from precomputed metadata costs less to measure than to model correctly.

The work-avoidance filter is itself a decision, not a fixed component. Applied unconditionally, the zone-map filter was a net loss across the corpus set – geometric mean **[geomean speedup, filter applied unconditionally]**, worst case **[worst-case speedup, filter applied unconditionally]**, and **[count of corpora harmed when the filter is applied unconditionally]** of **[total corpora in the gating comparison]** corpora harmed – while gating it on a cheap online test never lost and often won large (Fig. 4c). A filter only pays when it filters, and knowing when it filters is the same selection problem one level down. Selection at this scale is therefore no longer harmful, though it is not yet optimal: nothing measured lands within roughly **[residual factor from the bucket oracle, expected order approximately 1.5×]** of the bucket oracle.

FIGURE PLACEHOLDER

Selection cost against the runtime budget, regret against a bucket oracle, and gated-vs-unconditional filter speedup, by policy and host

Figure 4. Selection is cheap only when hoisted, and work avoidance must itself be gated. **a**, Selection cost as a percentage of total runtime for three policies – per-pair, per-tile, and probe-and-commit – on **[host count in the selection-cost sweep]** hosts. Dashed line marks the **[selection-cost budget, as a fraction of runtime]** runtime budget; hatched bars fail it. **b**, Regret of each policy against a bucket oracle that times every candidate cell in bulk over pairs grouped by shape and keeps the per-bucket minimum – an unattainable, non-deployable denominator, and a measured lower bound on true per-pair regret (Methods, "Selection policies: per-pair, per-tile, and probe-and-commit"). Hatched bars (all-bitmap; the per-pair cost model) are shown for scale, not as deployment candidates. **c**, Speedup of the zone-map work-avoidance filter over the unfiltered kernel across **[count of corpora in the gating comparison]** corpora (sorted by unconditional-filter speedup), applied unconditionally (grey) versus gated by the same selection machinery as **a** (green). Dashed line at 1.0×; annotated counts give the number of corpora harmed under each policy.

The winning representation pairing migrates across real corpora

The sweeps above vary density and working-set size by construction, and a generator can always be suspected of encoding exactly the structure its own kernels are built to exploit. We therefore repeated the central comparison on corpora of independent provenance: the twelve corpora CRoaring's own benchmark harness runs by default[8,9], together with five larger modern graphs, for 17 corpora spanning 7.89×10^{-7} to 1.73×10^{-1} in density and 1.995×10^{5} to 3.697×10^{7} bits in universe size (full provenance, and the further corpora scored only against an all-bitmap baseline, in Supplementary Table S1, Supplementary Note 6). Every corpus is scored against the same `run_optimize()`-tuned CRoaring baseline used throughout Results, so this comparison is continuous with the synthetic sweeps above rather than separate from them.

Table 1 reports, for every corpus, universe size and density, the pairing cell that won under best-cell (oracle) selection, and the ratio against tuned CRoaring and against an all-bitmap baseline. Both ratios are oracle: hindsight choice of representation pairing with no selection cost charged, not the deployed selector run end to end (Methods, "Selection policies: per-pair, per-tile, and probe-and-commit"); an end-to-end evaluation of the deployed selector is not part of this study. Storm's best cell beat tuned CRoaring on 17 of 17 corpora tested, ratio range 1.12× to 16.06×, median ≈ 6.1×, excluding `uscensus2000` and `dimension_033`, whose repeats could not be pinned to the precision shown (Methods, "Statistics and aggregation"; their density and winning cell are unaffected and still reported).

More informative than either ratio is which cell wins where. Across the corpus set, B×B wins 10, B×S wins 3, B×R wins 2, S×S wins 1, and R×R wins 1: no cell accounts for a plurality, let alone all of the corpora. Had one cell dominated, the pairing matrix would have had nothing left to select among; instead the winner tracks each corpus's density, as the density sweep above predicts. The sparse end is won by array- and rank-based cells, the dense end by B×B, and clustered corpora by run-based cells – the same three regimes the U-shaped curve predicts, now read off independently sourced data rather than a generator. The fill-compressed cell, W×W, underperforms tuned CRoaring on 16 of 17 corpora, the clearest real-data instance of the cost of choosing wrong that this paper argues throughout. Taken together, the migration and the histogram, not any single ratio, are the result reported here: a map of which representation is correct for which corpus, read directly off data whose structure this project did not generate.

A storage-priced container rule leaves query cost unclaimed

Several corpora at moderate-to-high global density – 1.2×10^{-3} to 1.7×10^{-1} – lose to Storm by more than an equally dense popcount workload should explain (Table 1), so we asked whether CRoaring's own container choice, not only Storm's kernels, accounts for the gap. We used CRoaring's own introspection API to census the array, bitset and run containers a `run_optimize()`-tuned bitmap settles on, counted after tuning so the census reflects how the library is actually deployed (Methods, "Container census").

On `census1881` at 1.2×10^{-3} global density, the tuned bitmap holds 1,332 array containers against **0** bitset and 132 run containers, out of 1,464 chunks (Table 2): essentially no chunk ever crosses the threshold, despite the corpus being dense by any global measure. The threshold tests a per-chunk statistic – a chunk's own cardinality – rather than the corpus's global density, so mass can spread across enough chunks that every individual chunk stays under threshold while global density reaches 6.3% (`weather_sept_85`) or 17.3% (`census-income`), and CRoaring runs array merges where a bitmap popcount would be strictly cheaper. This is not evidence of poor engineering; it is evidence that a threshold fixed once, at ingest, for a workload built around a single pairwise operation cannot be correct at every global density a corpus is later evaluated at – the

Table 1. The winning representation pairing migrates with corpus density. 14 of 23 real corpora (Methods, "Corpora, provenance and loaders"; full inventory in Supplementary Table S1), grouped by density band and chosen to span every domain and both speedup extremes; the remaining 9 corpora are summarised in the final row. `1KGP3 chr20` (haplotype-major) is included in the Mixed band as this paper's own stated counterexample and is not omitted for being unflattering. Both ratio columns are oracle: best-cell selection with hindsight, no selection cost charged, not the deployed end-to-end selector (Methods, "Selection policies: per-pair, per-tile, and probe-and-commit"). "vs. CRoaring" is against a `run_optimize()`-tuned baseline[8,9]. "vs. all-bitmap" is context only, from a separate benchmark, and the two ratio columns in one row need not come from the same measurement. Winning cell in bold, and the vs. CRoaring ratio is bold on the same rows, marking the rows for which a confirmed oracle result exists; —, not applicable. † ratio not quotable to the precision shown (Methods, "Statistics and aggregation"); density and winning cell are corpus properties and unaffected. ‡ vs. all-bitmap only; no CRoaring value exists for this row. § scoping corpus, not part of the 23-corpus set; a ratio below 1.0× is a genuine loss for selection at this density and layout. [All values provisional, pending the final measurement campaign.]

	Corpus	Domain	Universe m (bits)	Density	Winning cell	vs. CRoaring (oracle, ×)	vs. all-bitmap (×)
Sparse	`enwiki-categorylinks`	IR	1.30e8	7.89e-7	**[cell]**	**[n/a]**	1083‡
	`uscensus2000`†	census	3.70e7	8.09e-7	B×S†	†	†
	`dbpedia-link`	graph	1.83e7	1.63e-6	**[cell]**	**[n/a]**	311‡
	`as-skitter`	graph	1.70e6	9.08e-6	**B×S**	**4.02**	756
	`wiki-Talk`	graph	2.39e6	2.17e-5	**B×S**	**8.80**	910
	`gnomad_chr21_exomes_af1e-3`	genomics	1.46e6	1.93e-5	**[cell]**	**[n/a]**	222‡
	`dimension_008`	druid	3.87e6	8.98e-5	**R×R**	**1.73**	2846
Mixed	`usher_sarscov2`	genomics	8.45e6	1.87e-3	**[cell]**	**[n/a]**	121‡
	`wikileaks-noquotes`	text	1.35e6	1.02e-3	**B×B**	**6.79**	67
	`census1881`	census	4.28e6	1.17e-3	**B×B**	**16.06**	151
	`dimension_033`†	druid	3.87e6	5.78e-3	B×R†	†	†
	`1KGP3 chr20` (hap.-major)§	genomics	1.05e6	3.1e-2	**B×B**	**[n/a]**	0.93§
Dense	`weather_sept_85`	sensor	1.02e6	6.34e-2	**B×B**	**15.02**	2
	`msprime_10k`	genomics-sim	2.00e4	9.4e-2	**[cell]**	**[n/a]**	2‡
	`census-income`	census	2.00e5	1.73e-1	**B×B**	**13.11**	1
	remaining 9 corpora (Supplementary Table S1)		—	4.15e-6–7.9e-2	varies	1.12–6.07	3–523

Table 2. Roaring's fixed per-chunk threshold is tested against the wrong statistic. Container composition of a `run_optimize()`-tuned CRoaring bitmap[8,9], counted after tuning (Methods, "Container census"), for four corpora spanning the density range of Table 1, including `uscensus2000` at extreme sparsity to check whether the same array-container plurality persists there too (it does). Roaring assigns a bitset container to a 2^{16}-bit chunk only when that chunk's own cardinality exceeds a fixed compile-time constant (4096; vendor source, not measured here) — a per-chunk statistic, not the corpus's global density — so a corpus that is dense in aggregate can receive almost no bitset containers. Total chunks sums the three container counts. Bold marks the plurality container type per row; all four rows are plurality-array. [All values provisional, pending the final measurement campaign.]

Corpus	Global density	Array containers	Bitset containers	Run containers	Total chunks
`uscensus2000`	8.1e-7	**2,219**	0	2	2,221
`census1881`	1.2e-3	**1,332**	0	132	1,464
`weather_sept_85`	6.3e-2	**2,274**	561	21	2,856
`census-income`	1.7e-1	**553**	180	35	768

fixed-granularity limitation this paper's opening names.

That mechanism predicts that no single fixed cardinality threshold can be correct across the density axis, the same argument this paper makes generally, now applied to the incumbent. The constant itself points at why: 4096 is where a stored array and a bitset consume equal memory – a size crossover, chosen for storage rather than for compute – while the compute crossover for a count-only AND on this host sits at cardinality 64–128, so the stock threshold is 30–60× too high for this workload. We tested the prediction directly: promoting array containers above cardinality ≈64 to bitsets after `run_optimize()` – a twenty-line change to CRoaring – removes most of the dense-corpus advantage above (`census1881` 13.40× → 0.59×; `census-income` 10.65× → 1.88×; `wikileaks-noquotes` 5.15× → 3.67×), while several sparse corpora are unaffected or improve (`as-skitter` 3.35× → 3.23×; `dbpedia-link` 2.44× → 2.79×; `uscensus2000` 2.27× → 2.76×), for a net ≈ 19.5× gain on dense corpora against a 0.6–0.8× loss on sparse ones. Neither 4096 nor 64 is correct at both ends of the density axis: this is the paper's own thesis demonstrated inside the incumbent, and it reframes the finding above from a defect in Roaring to a case for adaptive selection in Roaring, for the same reason Storm needs it.

What this withdraws is a kernel-level advantage – Storm's dense cells beating CRoaring's dense cells – which was never the paper's contribution. With the avoidance mechanisms above (zone maps, gating, the size bound) applied on top of either CRoaring configuration, representation selection still wins, because those mechanisms sit above whichever kernel is underneath them and compose with a better one exactly as they compose with a worse one. The array-promotion fix itself is offered upstream rather than scored against: it is a twenty-line change to a widely deployed library, not a criticism of its engineering.
[Re-derive every CRoaring column in Table 1 against the array-promotion baseline before submission, and report both baselines throughout.]

Where the method does not pay

Although the best cell beat tuned CRoaring on 17 of 17 corpora tested above (oracle selection; Table 1), the method does not win everywhere, and its domain is worth stating precisely rather than implying it is universal.

The first boundary is the one drawn by the density sweep earlier in Results: between the two crossovers sits a band where nothing beats B×B, and 10 of 17 real corpora land there too. Inside that band the method's job is to recognise it cheaply and step aside, not to outperform a kernel that is already optimal.

The second boundary is that layout, not only content, determines which regime a dataset falls into. The same genomic cohort sits deep in the sparse regime in variant-major orientation – 121–222× against an all-bitmap baseline (`usher_sarscov2`, `gnomad_chr21_exomes_af1e-3`; no CRoaring comparison exists for either) – but in haplotype-major orientation lands inside the mid-band, where all-bitmap wins outright, 0.93× (`1KGP3 chr20`, hap.-major). The two orientations of the same underlying calls differ only in which axis is treated as rows and which as the shared universe (Supplementary Table S1); this paper does not choose the orientation a downstream dataset arrives in, and real data gives a large universe or sparsity far more often than both together.

The third boundary is the selector's own metadata. The rank index and zone map that make selection near-free are built unconditionally and scale with universe size rather than with the data they describe: on `enwiki-categorylinks` that is 4 MB of metadata attached to 155 bytes of chosen-representation data, a 26,312× overhead (Supplementary Table S2, Supplementary Note 7). Metadata built at ingest is meant to be amortised over a set's whole lifetime, the same assumption Roaring's own container choice makes (above); that amortisation pays only if the metadata is bounded, and here nothing yet declines to build it for a small, sparse row over a large universe. This is a known property of the current implementation rather than a design choice, and it makes the method undeployable on sparse, large-universe corpora regardless of speed until it is guarded.

The fourth boundary is the selector's own maturity: as established above, nothing measured in this campaign closes the residual gap to the bucket oracle, so selection here is cheap but not optimal. None of these four boundaries undermines the result reported above; together they define its domain – representation selection wins broadly, on a stated and bounded range of corpus shapes, with known and disclosed edges.

Discussion

This paper establishes that choosing which representation pairing to evaluate — cheaply, from properties measured online rather than modelled in advance — dominates the return available from accelerating any single kernel. A strategy built around avoiding work wins essentially every asymmetric measurement taken, and the resulting selector outperforms a competitively tuned Roaring baseline on every real corpus tested, with no single pairing winning enough of the corpus set to make the matrix itself unnecessary. The shape of the contribution mirrors Roaring's own: array containers, bitset containers, run containers and SIMD intersection were each individually known before Roaring existed, and what made the format matter in practice was their composition into one adaptively dispatched system, evaluated as a whole rather than argued for container by container[8,9]. The claim advanced here is structured the same way — a selector over a wider pairing matrix, gated on measured rather than modelled properties, is the contribution, not any cell inside it.

Roaring already performs adaptive, per-chunk representation choice, so the live question is not whether adaptivity helps but whether one fixed choice — a constant chunk width, a constant cardinality threshold, set once at ingest — can be right across the whole density axis a corpus is later evaluated at. This paper's own analysis says it cannot: a fixed threshold is tested against a chunk's own cardinality rather than the corpus's global density, so a corpus can be dense in aggregate while every individual chunk stays under the threshold (Table 2). Repairing that threshold inside the incumbent shifts the advantage in opposite directions at the two ends of the density axis, so neither constant is right and no single fixed threshold serves both regimes. The fix is offered here as a contribution to the incumbent, not a criticism of it — Roaring's default is a memory-sizing crossover, and it is exactly correct for the objective it was chosen for, which is not this one. What matters for the argument is that the work-avoidance layer keeps its advantage against the repaired baseline, because that advantage never depended on the kernel underneath being slow. Roaring needs adaptive selection across density for the same reason this paper's own selector does; that is the same argument, tested a second time, on someone else's code.

The mechanism behind the asymmetric-cell result is structural rather than incidental. Those cells are already work-proportional to the smaller operand, so a wider vector instruction has little left to accelerate; a rank lookup that answers an entire run without reading its bits, or a zone map that proves two rows disjoint from a fraction of their footprint, wins by removing work rather than by processing it faster (Fig. 3a). The complement carries equal weight: B×B is where work is genuinely irreducible in the generic case, and it is exactly there that two decades of vectorized popcount work[6] correctly pay off, including in this paper's own dense-cell kernel. Avoiding work and accelerating it are complementary axes, not competing ones, each correct on its own side of the boundary the density sweep locates. That boundary also responds to the memory hierarchy in opposite directions: vector throughput is a per-cycle constant a wider core keeps buying more of, realized fully only while data is fed at that rate, whereas work avoidance removes a count of memory-hierarchy crossings that widening a core does not by itself reduce (Fig. 3b). If core width and the ratio of cache capacity to DRAM bandwidth continue to diverge as recent processor generations suggest, the boundary measured here should move further toward avoidance rather than stay where it is — a prediction about future hardware, not a claim about the hardware tested in this study.

The property doing the work throughout this paper is heterogeneity, not sparsity. A corpus in which every row sits at the same density has nothing for a pairing matrix to exploit, however sparse or dense that shared density is; what representation selection is paid for is variance in density across rows that share one universe and one cost model, of which sparsity is only the more familiar half — the dense tail rewards exactly the same reasoning, mirrored around the complement. Read this way, the same argument should transfer to any workload that mixes elements of very different density under a shared representation and a shared cost model: sparse linear-algebra format selection among dense, compressed-sparse-row and coordinate tiles, or a columnar query engine deciding which blocks a predicate can skip without scanning, are settings where the condition plausibly holds. Neither has been tested here, and the claim is offered as where the mechanism should generalize, not as a result obtained in either domain.

Several things remain open, stated plainly. The selector is cheap relative to the saving it protects, but it is not close to optimal: nothing measured here closes the residual gap to a bucket oracle that already knows the winning cell (Fig. 4b). Every measurement reported here is single-threaded; the real-corpus comparison ran on **[one microarchitecture]** while the synthetic cross-architecture comparison spans **[four]**; and the tile pipeline has so far been evaluated on within-tile pairs only, with the cross-tile case and the transpose-build cost left to the end-to-end evaluation that follows this paper. Most directly, the pairing matrix, the selection gate and the tile pipeline described here are not currently reachable through the project's public C ABI: this paper reports a systematic measurement of the underlying mechanism, not a released tool, and the ABI and packaging work needed to close that gap is sequenced immediately after it, with a genome-wide linkage-disequilibrium application, Tomahawk[12], as the eventual, and strictly later, use of the mechanism rather than its motivation. What should outlast any single ratio in this paper is the map itself — which representation pairing wins where, and why — because that map is transferable independent of the number attached to any one cell.

Methods

What makes work avoidable

This subsection is an existence argument depending on no measurement: a gap must exist between what a dense-bitmap kernel *spends* and what a cheaper but equally exact algorithm on the same operands *already achieves*, and representations whose cost is a function of structure rather than of the universe can enter it. Both compared quantities are achieved; neither lower-bounds the work the problem demands, and no such bound is claimed here. Every representation is exact and interconvertible — B, S, R, W and the complement view C encode the same set and every cell returns the same cardinality — so nothing is approximated and only the cost changes. Every quantity below is a count of stored items or of operations, never a time; the constants that convert one to the other are measured, not derived, and are given under "Benchmark protocol and cache-residency statement". Each result is stated here with its hypotheses; the proof, the worked numerical checks and the secondary propositions that support but do not themselves state a result are in Supplementary Note 8, for the results up to and including the size bound, and Supplementary Note 9, for the representation-choice optimisation problem that follows it.

Write $A, B \subseteq [0, m)$, $a = |A|$, $b = |B|$, densities $d_A = a/m$, $d_B = b/m$, word width w, vector width V words per operation. Complementation is an involution on density, $d \mapsto 1 - d$, so define the *effective sparsity* $s_X = \min(d_X, 1 - d_X) \in [0, \frac{1}{2}]$ and $s = \min(s_A, s_B)$.

What cardinality alone determines. Given only a and b — the cheapest metadata any selector holds — the answer is confined to

$$\max(0, a + b - m) \;\le\; |A \cap B| \;\le\; \min(a, b), \tag{1}$$

and every integer in the range is realised (Supplementary Note 8). Its width is the quantity the rest of this subsection turns on:

$$\min(a, b) - \max(0, a + b - m) \;=\; \min(a,\, b,\, m - a,\, m - b) \;=\; m \min(s_A, s_B) \;=\; m s. \tag{2}$$

The four terms are the sizes of the four lists one could enumerate — A, B, A^c, B^c — so the uncertainty is the size of the smallest, and one near-empty or near-full side collapses it alone. Above density one half the floor lifts off zero at $(d_A + d_B - 1)m$: at $d_A = d_B = 0.51$, two per cent of the universe must overlap.

The fixed cost, and therefore the headroom. A dense bitmap stores $\lceil m/w \rceil$ words whatever it contains, so B×B performs $\lceil m/(wV) \rceil$ vector AND-plus-popcount operations per pair, independently of a, b, of structure, and of the answer. Against that, suppose each operand X is available as a bitmap and as a sorted array of whichever of X, X^c is smaller — $\Theta(ms_X)$ of space, the choice made from a, b, m in $O(1)$; no rank index is needed for this result. Then $|A \cap B|$ is answered in $O(ms)$ membership probes (four cases, one per term of Eq. (2); Supplementary Note 8). Counting descriptor items touched, and taking $w \mid m$,

$$\frac{\text{items touched by B×B}}{\text{items touched by the cheapest list-based cell}} = \frac{\lceil m/w \rceil}{ms} = \frac{1}{ws}, \tag{3}$$

which exceeds one whenever $s < 1/w$ and grows without bound as either operand leaves density one half — the headroom, derived rather than observed, and a comparison of two *achieved* costs, not a lower bound on any algorithm's work. Two of the four cases need a complement view, which is why C is part of the design rather than an afterthought for the dense tail. Charging measured per-item costs — a random probe against a vector word — turns it into a crossover in s of **[measured crossover in effective sparsity]**.

From descriptor length to operation count. Everything else here counts *stored items*, a fact about representation size; the paper's claim is about work. One identity carries that step. Let A be a bitmap with an $O(1)$ rank index, $\mathrm{rank}_A(i) = |A \cap [0, i)|$, and B a run container with maximal runs $\{[u_i, v_i)\}_{i=1}^{r}$. The runs partition B, so

$$|A \cap B| = \sum_{i=1}^{r} \big(\mathrm{rank}_A(v_i) - \mathrm{rank}_A(u_i) \big), \tag{4}$$

in exactly $2r$ lookups — independent of m, of $|A|$, of $|B|$, and of the run lengths (Supplementary Note 8). A run of any length is priced as a run of length one, because the kernel never looks inside it; symmetrically R×R by merge costs $O(r_A + r_B)$ with no index. This is what makes a run count a cost parameter and not merely a storage parameter, and it is the only bridge between the two here: it reaches *operation count*, not time, since per-operation constants still differ across cells.

What randomness would predict. Let each position be set independently with probability d, and write $\kappa(\rho)$ for the number of stored items representation ρ must touch. The first two entries below are exact; the last two are the leading-order forms of exact Bernoulli expectations (Supplementary Note 8), written in $s = \min(d, 1-d)$,

$$\mathbb{E}\,\kappa(\mathsf{B}) = \left\lceil \tfrac{m}{w} \right\rceil, \quad \mathbb{E}\,\kappa(\mathsf{S}) = md, \quad \mathbb{E}\,\kappa(\mathsf{R}) = md(1-d) = ms(1-s), \quad \mathbb{E}\,\kappa(\mathsf{W}) = \tfrac{m}{w}\big[1 - (1-s)^{2w} - s^{2w}\big], \tag{5}$$

where $\kappa(\mathsf{R})$ counts maximal runs and $\kappa(\mathsf{W})$ literal words plus fill groups. None of these expectations is new. The W row is the word-aligned expected-size model of Wu, Otoo and Shoshani[13], whose bracket is identical to the one above under their payload width of $w-1$ bits per word rather than w; the symmetry about $d = \frac{1}{2}$ and the resulting incompressibility at that density are theirs as well. The R row is the classical expected run count for a uniformly random arrangement[14], and the interval bound used below is the Fréchet–Boole inequality. The density–clustering parameterisation is also due to[13]; the design space it spans has since been mapped empirically for Roaring, WAH and tree-encoded bitmaps by Lang et al.[15], who report that an uncompressed bitmap reads faster than any of the three compressed formats over $16 \le f \le 128$ and $0.01 \le d < 1$, and that they expect "a performance-optimized implementation to dominate an even larger space" than their unoptimized baseline did. That is the headroom this paper measures, stated from the compressed side. They are restated here in one notation because the pairing matrix needs them as inputs, not because they are results of this paper. With the complement view making the best of S and C∘S cost ms, every entry is constant in s or strictly increasing on $[0, \frac{1}{2}]$ and symmetric about $d = \frac{1}{2}$: under this null model the U-shaped cost curve is a theorem — about the model and the bitmap's flatness, not about any method. Randomness gives run containers nothing either, since $\frac{1}{2}ms \le \mathbb{E}\,\kappa(\mathsf{R}) \le ms + 1$, so choosing *among* {S, C∘S, R, W} buys at most a constant on structureless data. B is the exception and the exception is the point: $\kappa(\mathsf{B})$ is $\lceil m/w \rceil$ at every density, so the spread across the full matrix is Eq. (3) and unbounded. It is the envelope, not each cell, that follows $\Theta(\min(m/w, ms))$.

The deviation, which is the contribution. Real data is not random, and the representations part company with Eq. (5) in a way density cannot express. At fixed density $d \le \frac{1}{2}$ with $k = dm$, $\kappa(\mathsf{B})$ and $\kappa(\mathsf{S})$ are constants, while $\kappa(\mathsf{R})$ ranges over every integer in $[1, k]$ and $\kappa(\mathsf{W})$ from $O(1)$ to $\Theta(\min(k, m/w))$ (Theorem B, proved by explicit families in Supplementary Note 8). Writing $\sigma(\rho, X)$ for predicted descriptor length at X's density over actual, σ is identically one for B and S, at least

$\max(d, 1-d+\frac{1}{m}) \geq \frac{1}{2}$ for R, and unbounded above: real rows can be arbitrarily cheaper than random rows of the same density and at most about twice as expensive.

One impossibility follows, exactly. A selector reading density — or cardinality, or any function of them — evaluates one fixed function on every row of that density, so by the range above it mis-predicts $\kappa(\mathsf{R})$ by an unbounded factor on some rows at every density; a fixed representation cannot exploit σ either, having committed before σ is knowable. That rules out a family of selectors without choosing among the survivors, and two further readings do not follow: whether a mis-predicting model costs more than selecting nothing depends on which cell it picks and that cell's actual cost, which is measured (Fig. 4b), not derived; and a run or occupancy count is $O(1)$ metadata that sees σ perfectly and could feed a model. What is proved is that selection must be gated on a statistic that sees structure rather than on cardinality — structural summary or direct kernel measurement is settled below, by measurement.

The null model is a floor, and it is the same floor three times. Independent placement at fixed density is the *extremal* arrangement for every mechanism used here, always in the same direction (proofs in Supplementary Note 8). For a representation, Eq. (5) sits within a factor of two of the worst achievable descriptor. For a zone map of bin width W_z, scattering maximises the occupied-bin fraction, so the summary's null cost is an upper bound on its cost and a lower bound on its value. For the prefix filter of the narrowed-contract mode, the entire mechanism depends on the operands through one dimensionless quantity, and any ordering that concentrates rare elements into prefixes makes real prefixes collide less often than scattered ones of the same length. In each case the null is what the mechanism is worth on data with no structure to find, real data deviates from it only in the mechanism's favour, and that deviation is exactly the quantity the mechanism converts into avoided work.

Where the headroom vanishes. At $s = \frac{1}{2}$ the interval has width $m/2$ and excludes nothing, the cheapest list-based cell spends $m/2$ probes against $\lceil m/w \rceil$ words, and Eq. (5) offers neither R nor W an advantage: Eq. (3) falls below one. In the generic case the mid-density band belongs to $\mathsf{B}\times\mathsf{B}$, and a selector choosing it there is correct rather than defeated. This bounds what cardinality determines absent structure, not what any kernel must spend — call it a *metadata* or *identifiability floor*, never an information-theoretic one — and a structured row at exactly this density is a counterexample to the unqualified form (Supplementary Note 8). It is also why the zone map of the next paragraph matters in the one band where cardinality is useless.

Why the two mechanisms are selected between rather than stacked. Composing a zone map with a representation is not the product of the two, because filtering *concentrates* what it passes on: conditioned on a bin of width W_z being occupied its expected local density is d/p, $p = 1-(1-d)^{W_z}$, not d — no summary hands a kernel anything sparser than one bit per bin (Supplementary Note 8). Charging the summary pass c/W_z of a bitmap scan and taking the operands independent, the composed cost relative to a plain bitmap is

$$\frac{c}{W_z} + p_A\, p_B\, \kappa\Big(\frac{d}{p}\Big), \tag{6}$$

$\kappa \leq 1$ being the envelope of Eq. (5) normalised against the bitmap. The composition never costs more than the summary alone, yet is floored at c/W_z, below which a sparse representation is cheaper outright — three regimes, the middle one exactly $xe^{-x} > c/w$ in $x = W_z d$ bits per bin, opening where the summary undercuts enumeration and closing where bins stop being empty. This is the analytic form of the gating result of Fig. 4c, and it models *work*: it bounds where a filter has work to remove and says nothing about what one costs when it removes nothing.

Each mechanism's liability is two-sided, for unrelated reasons at the two ends. A summary is indexed by the universe while every compressed representation is indexed by the data, so their costs must cross. Charging the summary c/W_z of a bitmap scan as in Eq. (6), and a one-word-per-element representation wd, the low crossing is

$$d_{\text{low}} = \frac{c}{wW_z} = 3.1\times10^{-5} \qquad (c=1,\ w=64,\ W_z=512), \tag{7}$$

below which reading the summary costs more than enumerating the operands outright (Supplementary Note 5). At the other end $p_A p_B \to 1$: no bin is empty, the summary proves nothing, and only its own c/W_z remains. A run- or fill-based representation crosses at its own density, wherever its descriptor length falls below m/W_z, but that a crossing exists at all is generic. Prefix filtering has the same shape in $y = p^2/m$: for $y \ll 1$ the surviving fraction is y itself, so pruning improves quadratically as prefixes shorten, while for $y > 1$ prefixes collide by default and the index earns nothing. This is why no mechanism here is applied unconditionally, and why each gate is a two-sided test rather than a threshold.

A narrowed contract, and what two integers then settle. Every statement above returns the cardinality itself. A caller may instead ask a narrower question — report only the pairs with $J(A,B) \geq t$, or only those with $|A\cap B| \geq \tau$, or restrict attention to

operands whose size falls in a band — and the narrowing is itself exploitable, because Eq. (1) already says what sizes permit:

$$J(A,B) \;=\; \frac{|A\cap B|}{|A\cup B|} \;\le\; \frac{\min(a,b)}{\max(a,b)}, \tag{8}$$

so $J(A,B)\ge t$ requires $\min(a,b)\ge t\,\max(a,b)$, and $|A\cap B|\ge\tau$ requires $\min(a,b)\ge\tau$ (proof in Supplementary Note 8). The hypotheses are that $a,b>0$ and $t\in(0,1]$; nothing is assumed about arrangement, about m, or about how either operand is stored. Both conditions are necessary and neither is sufficient, so the test discards pairs and never reports one — the survivors still go to an exact kernel, at the cost of one comparison between two integers already held as metadata, no element of either set read. This is the Swamidass–Baldi pruning bound[3] together with the size (length) filter of[16], in the two-sided form standardised by[17] and applied jointly with prefix filtering to Tanimoto over binary vectors by[18], restated here to place it against the mechanisms above rather than as a result of this paper.

What that removes, and why a wider corpus helps it. Modelling a corpus by drawing cardinalities independently with $\log a$ uniform on $[0,\ln K]$ — the density $\Pr(a)\propto 1/a$, exactly the $1/i$ cardinality spectrum this paper's benchmark protocol mandates ("Corpora, provenance and loaders"), not an approximation to it — $\min(a,b)/\max(a,b)\ge t$ becomes $|\log a-\log b|\le\gamma$, $\gamma=\ln(1/t)$, and for two independent uniforms on an interval of length $L=\ln K$,

$$\Pr\big[\text{pair can qualify}\big] \;=\; 1-\Big(1-\frac{\ln(1/t)}{\ln K}\Big)^2, \qquad \ln(1/t)\le\ln K, \tag{9}$$

with the absolute form $\big(1-\ln\tau/\ln K\big)^2$ (proof and worked values in Supplementary Note 5, which agree with a 10^5-sample simulation to three decimals). Expanding for small γ gives $2\gamma\int\phi^2$ for any density ϕ of $\log a$, so the governing quantity is the *effective log-spread* $1/\int\phi^2$ of the size distribution, of which Eq. (9) is the log-uniform case. Two hypotheses carry real weight. Cardinalities are integers, and $a=b$ passes at every t, so the continuous form is optimistic where sizes are small. And the favourable scaling in K — Eq. (9) decreases monotonically in K — is a property of the log-uniform family and not of heavy tails generally: under the alternative reading in which *ranked* sizes scale as $1/i$, the surviving fraction is exactly $1-t$, independent of K, since the effective log-spread saturates at two nats however far the sizes are spread. The favourable scaling in K is therefore claimed for the spectrum this paper benchmarks, not for every skewed corpus.

The size bound does not multiply with the mechanisms below it. Acting at a coarser granularity is not sufficient for savings to compose. The size test decides whether a pair is formed at all, while representation choice and the zone map act inside a pair that is formed, so the granularity argument of Eq. (6) predicts a product — and the product is not achieved, because both mechanisms read the same statistic. The survivors of Eq. (8) are the pairs with $a\approx b$, and every mechanism below is cheapest exactly when a and b are *dissimilar*, since its cost tracks $\min(a,b)$ against a fixed $\lceil m/w\rceil$. Conditioning raises the mean cost of a survivor by

$$\frac{\mathbb{E}\big[\min(a,b)\;\big|\;\min/\max\ge t\big]}{\mathbb{E}\,\min(a,b)} \;=\; \frac{L^2\big(K(1-t)-\gamma\big)}{\gamma(2L-\gamma)\,(K-L-1)} \;\xrightarrow[t\to 1]{}\; \frac{\ln K}{2}, \tag{10}$$

and the composed work exceeds the product of the two savings by that same factor; the derivation, the numerical verification and the decorrelation control that isolates the shared statistic as the cause are in Supplementary Note 8. The composition is still strongly worth taking. What is refuted is only the product; the general rule is that mechanisms compose multiplicatively when they act on different waste *and* are driven by different operand statistics, and sharing a statistic costs the composition Eq. (10)'s factor even across granularities. Values here are analytic, computed over the model above at $m=10^6$, and count work rather than time.

Choosing a representation is a different problem from choosing a size, and only one of them is separable. Everything above prices representations; a deployed library must *choose* between them, and the choice has to be stated as an optimisation before it can be called optimal. Partition the universe into chunks of M bits, each stored in one representation, $n=M/w$ words to a chunk and e bits to a stored array element. Given a corpus, a distribution π over the chunk pairs an application actually evaluates, and an operation ω, the problem is to choose an assignment ρ of representations to chunks minimising $\sum_{(x,y)\sim\pi} C_\omega(\rho(x),\mu_x;\rho(y),\mu_y)$ subject to a byte budget, where μ is the metadata of Eq. (5) and C_ω counts items touched at measured per-item constants. Storage is a *separable* objective — minimised one container at a time — whereas the query cost is quadratic in the assignment, since there is no such thing as the cost of a container, only the cost of a pair (Supplementary Note 9). And the incumbent's three container decisions — array against bitset, array against run, bitset against run — are one rule, arg min over serialized size, so its array-to-bitset threshold is $\tau_{\text{stor}}=M/e$, a property of the format carrying no machine constant and no property of the workload: the exact optimum of a separable objective, and why it can be a compile-time constant. What follows is what changes when the objective is not separable.

The crossover is a formula, and the two constants in the field are its two limits. Write c_w for a vectorised word AND-plus-popcount, c_p for a membership probe into a bitmap and c_m for a merge step, and $\theta_{pw} = c_p/c_w$, $\theta_{mw} = c_m/c_w$. Promotion of a chunk of cardinality k from array to bitset is favourable against a bitmap partner exactly above

$$\tau_\cap \;=\; \frac{c_w}{c_p}\cdot\frac{M}{w} \;=\; \frac{n}{\theta_{pw}}, \qquad \text{whence} \qquad \frac{\tau_{\text{stor}}}{\tau_\cap} \;=\; \frac{w}{e}\,\theta_{pw}, \tag{11}$$

and against an array partner above $(\theta_{pw}/\theta_{mw} - 1)\,k_Y$ for a partner of size k_Y (proof in Supplementary Note 9). Pricing a byte at $\lambda \geq 0$ and minimising query cost plus λ bytes gives a threshold that runs from $\tau_\cap$ at $\lambda = 0$ to τ_{stor} as $\lambda \to \infty$: every value between the two is optimal at some price of a byte, and $\tau_{\text{stor}} = M/e$ is the unique threshold at which promotion is never paid for in bytes. The threshold also carries the pair count, so under any byte budget the optimum is not a function of the container at all. The measured input is the symmetric crossover **[cardinality at which an array–array merge stops beating a bitmap–bitmap popcount]**, which fixes θ_{mw}; substituted into Eq. (11) at $M = 2^{16}$, $w = 64$, $e = 16$ together with **[measured ratio of a bitmap probe to a merge step]** it gives **[the ratio of the two thresholds]**, recovering the divergence between the two objectives from measurements of the constants and no free parameters. Both crossovers share c_w, so measuring them together over-determines the model and the probe-to-merge ratio becomes a checkable prediction rather than an input.

A fixed threshold is optimal, under five hypotheses, and each of them fails. The loose form is false, so state the hypotheses first: if the candidates are only S and B, the operation is a count-only intersection, the per-item constants do not depend on the resident footprint, there is no byte budget, and every chunk faces the same partner mix, then the optimal assignment *is* the fixed threshold $\tau_\cap$ of Eq. (11) — a machine constant, independent of the corpus. Dropping any one hypothesis breaks it, with an exact witness for each (Supplementary Note 9): admitting R reintroduces the impossibility above, specialised to the container decision, since R×B costs a strictly increasing function of run count while S×B does not depend on it; letting the partner mix vary makes the optimal threshold

$$\tau^*(\beta, \bar{k}) \;=\; \frac{\beta\, c_w n + (1-\beta)(c_p - c_m)\,\bar{k}}{\beta\, c_p + (1-\beta)\, c_m}, \tag{12}$$

where β is the pair-weighted fraction of a chunk's partners that are bitmaps and $\bar{k}$ the mean cardinality of the array partners: a statistic of the workload rather than of the machine, and self-referential — an optimal fixed threshold is a fixed point $\tau = \tau^*(\beta(\tau), \bar{k}(\tau))$, which exists but need not be unique; admitting a byte budget makes the threshold its dual variable, sweeping the whole interval above; changing the operation flips the matrix's sign, since cardinality queries have no operation dimension at all (union, difference and symmetric difference are inclusion–exclusion over the intersection) but a *materialising* union with a bitmap operand costs n words whatever the other side is, so no single stored assignment serves a mixed workload. The fifth hypothesis — that the per-item constants do not depend on footprint — is the one that is not analytic and the one the measurements say dominates: c_w is not a constant, footprint re-enters through it, and a work model that prices the choice cannot price its consequence, the same divergence recorded under Eq. (6) and treated as a measurement question, not an analytic one.

Metadata settles the decision wherever the decision matters. Two identifiability questions must not be merged. Eq. (2) says what cardinality determines about the *answer*, pinned only at the density extremes. What cardinality determines about the *decision* is different and more favourable: every cost in this paragraph and the two before it is a closed form in $(k, r, \ell + g)$ and the constants, so a selector holding those three integers per operand evaluates the ordering exactly, in $O(1)$, touching no operand data — which is what "selection must be near-free" means, and a consequence of the cost model rather than an engineering aspiration (proof in Supplementary Note 9). Holding only k, the ordering is not determined once R is a candidate; one extra integer restores it and more cardinality precision does not. Suppose the constants are known within a factor $\eta \geq 1$ — the uncertainty of the modelled costs, distinct from the γ of the size bound above — and let Λ be the ratio of the two cheapest modelled costs: if $\Lambda > \eta$ the ordering is determined; if $\Lambda \leq \eta$ it is not, and choosing the second-best costs at most η. So for every pair, either the metadata settles the decision or the decision does not matter to within the precision of the constants, and the undetermined set is an η-neighbourhood of the boundaries rather than a region of unbounded error. The caveat that makes η interesting: it is the uncertainty of the *model*, and where the model omits the memory system it is not small — the derived reason to measure a candidate rather than only to model it.

Why an advantage survives repairing the kernel beneath it. A pairwise batch costs C_{meta} plus the cell cost summed over the pairs a summary does not discard and the regions it does not exclude; those sets are functions of cardinality, bin occupancy and the data, and of nothing else, so no change of representation moves either set (Supplementary Note 9). Consequently, if a better kernel divides every cell cost by α, the advantage over the same batch without avoidance is invariant in α when the summary is free and erodes only through the summary's share of the surviving cost: improving a kernel cannot subsume avoidance, because avoidance changes which work is asked for and a kernel changes only what asked-for work costs. Which

savings multiply is settled by the same rule as Eq. (10): a zone map reads bin occupancy while representation choice reads cardinality and run count, so those multiply; the size bound reads cardinality, which representation choice also reads, so those do not, and a container-threshold gain must not be multiplied into a size-bound gain. Quantitatively, against an incumbent already holding the compute-optimal threshold of Eq. (11), the modelled advantage of adding a zone map of bin width W_z peaks at

$$G_{\max} = \frac{\theta_{pw}\, w}{2\sqrt{c\, W_z}} \qquad \text{at} \qquad d^* = \frac{\sqrt{c}}{W_z^{3/2}}, \tag{13}$$

valid on $(\theta_{pw} w\sqrt{c})^{2/3} \le W_z \le \theta_{pw} w$ and saturating at $\sqrt{\theta_{pw} w/c}\,/2$ above it, and falling below one outside a window whose lower edge is Eq. (7) with the per-item constants carried, $c/(\theta_{pw} w W_z)$ (proof in Supplementary Note 6). Eq. (13) is a *null-model* ceiling on structureless data, and by the one-sidedness of σ above, real corpora can only fall below the null, so a measured advantage exceeding it is evidence of structure and must be reported as such rather than as agreement. And Eq. (13) excludes the narrowed contract entirely: Eq. (9) supplies its own factor, and by the rule above the two are not multiplied.

Where avoidance is unavailable, throughput is what remains. The mid-band above is not only where these results stop applying; it is where a different optimization applies, and the two are complementary rather than competing. Where Eq. (3) falls below one and $p_A p_B \to 1$, the work $\mathsf{B}\times\mathsf{B}$ performs is, in the generic case, work that is going to be performed, and what remains to optimize is the per-cycle throughput of its $\lceil m/(wV)\rceil$ vector `AND`-plus-popcount operations — a quantity none of the results above speaks to, and one the popcount literature has already driven close to the hardware limit[6]. Eq. (3) and Eq. (6) locate the boundary between the two domains; they do not rank them.

Representations and the per-cell cost model

Each row — one set X_i over a shared universe of m bits — is stored in one of several representations, each with a distinct asymptotic cost for computing an intersection *cardinality*, $|X_i \cap X_j|$, rather than the intersection itself. We define each representation below, together with the cost of the cell it forms when paired with itself or with another representation; every cost-model symbol used elsewhere in this paper is defined here and only here.

Four representations form the symmetric pairing matrix. A dense bitmap (B) stores m bits directly, one per universe position, and costs $\Theta(m)$ to intersect against another dense bitmap by AND and popcount, independent of either side's cardinality. A sorted array (S) stores the set's $|X|$ set positions as sorted integers and costs $\Theta(|X|)$ against a bitmap, since each element is resolved by a single random probe. A run container (R) stores the set as r maximal runs (a,b) of contiguous set bits and, against a bitmap fitted with a rank index (below), costs $\Theta(r)$ — independent of run length. A fill-compressed word stream (W, an EWAH-style encoding of literal words interleaved with run-length-coded fill words) costs $\Theta(\ell+g)$ against a bitmap, where ℓ is the literal-word count and g the fill-group count, and its fills are skipped in constant time using the same rank index used for R. A fifth view, the complement (C), does not add a representation to the symmetric matrix; it is a strategy, described in the next subsection, for reusing whichever of B, S, Ror Wis cheapest on a row's complement when that row sits above density one-half.

The cost of each symmetric cell is, in the notation above: $\mathsf{B}\times\mathsf{B} = \Theta(m)$ (naive; made sub-linear by the zone-map filter below); $\mathsf{B}\times\mathsf{S} = \Theta(|S|)$; $\mathsf{B}\times\mathsf{R} = \Theta(r)$ with a rank index; $\mathsf{B}\times\mathsf{W} = \Theta(\ell+g)$; $\mathsf{S}\times\mathsf{S} = \Theta(|A|+|B|)$ by merge or $\Theta(|A|\log(|B|/|A|))$ by galloping search when $|A| \ll |B|$; $\mathsf{S}\times\mathsf{R} = \Theta(|S|+r)$ by merge or $\Theta(|S|\log r)$ by binary search into runs; $\mathsf{S}\times\mathsf{W} = \Theta(|S|+\ell+g)$; $\mathsf{R}\times\mathsf{R} = \Theta(r_A+r_B)$ by run merge; $\mathsf{R}\times\mathsf{W} = \Theta(r_A+(\ell+g)_B)$; and $\mathsf{W}\times\mathsf{W} = \Theta((\ell+g)_A+(\ell+g)_B)$. Every cell except $\mathsf{B}\times\mathsf{B}$ is sub-linear in the universe size m. The formal statement of what these costs imply — that the null model is a floor rather than a prediction, and the exact conditions under which no representation beats a bitmap — is given with full proofs in Supplementary Note 8.

The pairing matrix and its cells

The paper reports the ten cells of the symmetric representation matrix formed from {B, S, R, W}: $\mathsf{B}\times\mathsf{B}$, $\mathsf{B}\times\mathsf{S}$, $\mathsf{B}\times\mathsf{R}$, $\mathsf{B}\times\mathsf{W}$, $\mathsf{S}\times\mathsf{S}$, $\mathsf{S}\times\mathsf{R}$, $\mathsf{S}\times\mathsf{W}$, $\mathsf{R}\times\mathsf{R}$, $\mathsf{R}\times\mathsf{W}$, $\mathsf{W}\times\mathsf{W}$. The complement strategy, $\mathsf{C}\times\mathsf{B}$, is an eleventh, asymmetric strategy rather than a twelfth symmetric cell: when a row's density places it above one-half, its complement is computed and intersected using whichever of the ten cells above is cheapest for that complement, and the requested cardinality is recovered by inclusion–exclusion, $|A\cap B| = |B| - |A^c\cap B|$, where A^c is A's complement against the shared universe. This convention is used consistently throughout: any count of "ten" cells in this paper refers to the symmetric matrix over {B, S, R, W}, and $\mathsf{C}\times\mathsf{B}$ is reported separately as a strategy rather than folded into that count. Roaring is never constructed as a cell of this matrix; it is used throughout as an external baseline only (below).

Each cell is implemented independently against a shared representation layer providing one view per format (bitmap, sorted list, run, EWAH-fill and complement). Several of the asymmetric cells have more than one implemented strategy — for example $\mathsf{B}\times\mathsf{R}$ has a rank-indexed variant and a scalar per-run-scan variant — and each variant was raced against the independent oracle

described below before being admitted to any comparison. The variant reported for a given cell in Results is the one selected as fastest for the density regime or corpus shape being reported, not a single fixed implementation chosen in advance; the full record of implemented variants, including those that did not win any regime, is not itemized in this paper (Code availability).

Zone-map and rank-index construction

Two mechanisms let a kernel decide how much of a row it needs to visit, rather than visiting all of it faster. To decide how much of a dense row a pairing needs to scan, we used a *zone map*, a bitmap of coarser granularity in which bit k records whether bin k of the underlying row — a contiguous block of **[zone-map bin width in bits]** bits — contains any set bit. To answer a run's or a fill's contribution to a bitmap without touching the bits it spans, we used a *rank (prefix-popcount) index*, following the rank/select lineage of Jacobson, Clark and Vigna's `rank9`[19–21].

The zone map is built by a single forward pass over the row, or is free at query time when the underlying storage already carries block-occupancy metadata, as columnar formats commonly do; its storage cost is **[zone-map storage cost as a fraction of the bitmap it summarizes]** of the bitmap it summarizes. For a pair, the bitwise AND of the two rows' zone maps, $\mathrm{occ}_A \wedge \mathrm{occ}_B$, is itself a bitmap of $m/$**[zone-map bin width in bits]** bits, and its popcount is the *exact* number of bins the pairing needs to visit, not an estimate: a popcount of zero proves the two rows disjoint without visiting either. The rank index is built by a single forward prefix-popcount pass and stores **[rank-index storage cost as a fraction of the bitmap it indexes]** of the bitmap it indexes; for a run or fill spanning positions $[a,b)$ on the indexed side, its contribution to the intersection is answered in constant time as $\mathrm{rank}_B(b) - \mathrm{rank}_B(a)$, the number of set bits in B up to b minus the number up to a — this formula is used, unchanged, by every cell that pairs a run- or fill-based representation against an indexed bitmap.

Neither mechanism is applied unconditionally. The zone-map filter is consulted only ahead of cells that would otherwise scan a dense representation in full (B×B and B×W); it is not applied to B×S or B×R, which are already work-proportional to the sparse side and for which a summary can remove nothing a direct probe does not already skip. Applied to a row whose bins are uniformly occupied, the filter finds nothing to skip and its cost is pure overhead; the policy that decides, per pair, whether the filter is worth consulting is described as a selection policy in the next subsection, and its necessity is established as a result rather than assumed as a procedure (Fig. 4c). This is a re-parameterization of established occupancy-summary mechanisms rather than a new one: BitFunnel's hierarchical block index[10], the hierarchical bitmap index of[22], WAH/EWAH's own uniform-fill words, and Lucene's sparse-block bitset all skip empty regions by a similar test; what is specific to this setting is applying the same test as an *exact* work-count for intersection cardinality, where the summary's own popcount is the number of bins that must be visited rather than a heuristic estimate of it. The cost model for this filter, the crossings that bound where it pays, and the derivation of the bin width used here are given in Supplementary Note 4.

Selection policies: per-pair, per-tile, and probe-and-commit

Selection — choosing which cell to run for a given pair — is distinct from the zone-map and rank-index mechanisms above, which decide how much of an already-chosen cell's work can be skipped; the two are reported separately throughout. Three selection granularities were evaluated.

Under *per-pair* selection, precomputed $O(1)$ metadata for each row — cardinality, run count, and per-row zone-map bin-occupancy count — is consulted once for every pair (i, j), and the cell predicted cheapest under the cost model above is chosen. Under *per-tile* selection, rows are first partitioned into tiles of **[tile width in rows]** rows by sorting or bucketing on density and run count, so that rows sharing similar characteristics fall into the same tile; one cell choice is then made per tile, from the same $O(1)$ metadata aggregated over the tile, and applied to every pair inside it, hoisting the decision out of the innermost loop. Under *probe-and-commit*, named here as the $\varepsilon \to 0$ special case of Micro Adaptivity in Vectorwise[11], a small number (**[count of probe pairs sampled per tile before committing]**) of a tile's pairs are timed under each candidate cell before any decision is made, and the tile commits to whichever cell was empirically fastest on that sample for its remaining pairs. All three policies are evaluated against an explicit runtime budget of **[stated selection-cost budget as a percentage of total runtime, from the design target this paper adopts]** of total runtime; a policy that exceeds the budget is reported as failing it, not as a qualified pass.

Selection cost was measured by instrumenting the decision path itself separately from the kernel calls it dispatches to, so that the reported cost of choosing is not conflated with the cost of computing. *Regret* against a bucket oracle is reported alongside selection cost for every online policy: the oracle is the best cell for a given bucket of pairs chosen with full hindsight after every candidate cell has been timed on it, and a policy's regret is its runtime on that bucket expressed relative to the oracle's runtime on the same bucket. The same bucket oracle is used to report regret for an always-Bbaseline and for a closed-form per-pair cost model evaluated from the metadata above but without online timing, so that measured selection can be compared directly against both a no-selection baseline and a modelled-selection baseline on the same metric.

Two distinct oracles are reported in this paper and are never interchanged. The *bucket oracle* defined above operates at bucket granularity and is the denominator for every regret figure. The *best-cell oracle* reported as an upper-bound column in Table 1 is coarser: for a whole corpus it takes the single cell whose measured end-to-end runtime on that corpus is lowest,

chosen after the fact and charged no selection cost. Because it may not vary its choice within a corpus, it is not a lower bound on the bucket oracle's runtime; it is reported only as a bound on what a per-corpus selector could achieve, and the deployed-versus-oracle gap in Table 1 is regret against that bound alone. The choice these policies approximate is itself an optimisation problem with a characterisable optimum; it is stated and solved, with the conditions under which a metadata-only decision is sufficient, in Supplementary Note 9.

Corpora, provenance and loaders

Three corpus families were used. The first is the set of corpora CRoaring's own benchmark harness runs by default (the `real-roaring-datasets` collection), including `census1881`, `census-income`, `weather_sept_85`, `wikileaks-noquotes`, `uscensus2000` and the `dimension_0`*xx* family, together with the row-sorted (`_srt`) clustering variant of each that the original Roaring papers also report. The second is a set of larger modern graph corpora added beyond CRoaring's default set, drawn from public network-graph collections (including `com-LiveJournal`, `com-Orkut` and `as-skitter`). The third is genomic: two population-scale variant call sets, UShER SARS-CoV-2 and gnomAD chromosome 21, both stored variant-major (one row per variant site, one bit per sample), and one haplotype-major encoding of 1000 Genomes Project phase 3 chromosome 20 (one row per haplotype, one bit per site) used specifically to demonstrate that storage layout, not the underlying data, determines whether a corpus lands in the sparse or the mid-density regime. Universe size and density are reported per corpus in Table 1 and are not duplicated in prose here; every corpus is drawn against the same competitively tuned CRoaring baseline used throughout Results.

As a loader-correctness check, the `census1881` loader was validated by reproducing the published corpus statistics reported for it (universe size and mean row cardinality) against the values reported in the Roaring bitmap literature[23], rather than trusting the loader's own output uninspected.

Container census

The container counts in Table 2 are read from CRoaring's own introspection API, `roaring_bitmap_statistics()`, and not inferred from our measurements. The census is taken after `run_optimize()` and `shrink_to_fit()`, so that it reports the containers the library settles on once it has had the opportunity to choose run containers, matching the tuned baseline every ratio in this paper is computed against. Four fields are recorded per corpus: `n_array_containers`, `n_bitset_containers`, `n_run_containers`, and the total container count. The threshold governing that choice is a vendor fact rather than a measurement of ours: CRoaring stores a 2^{16}-bit chunk as an array container unless the chunk's own cardinality exceeds the compile-time constant `DEFAULT_MAX_SIZE`, whose value is 4096 (declared as `enum { DEFAULT_MAX_SIZE = 4096 }` at `roaring.h:2486`, and applied at insertion by `array_container_try_add(ac, val, DEFAULT_MAX_SIZE)` in `container_add`, `roaring.h:5690`, in the pinned version below). The constant is not a tuning knob: it is an `enum` with no override, it is enforced as a structural invariant by `array_container_validate` and `bitset_container_validate` (`roaring.c:7133`, `roaring.c:8263`), and the portable serialization format writes no array-versus-bitset type tag at all, reconstructing the type from cardinality against this same constant on read (`roaring.c:14421`, `roaring.c:14553`). The same fixed commitment prices a run container against a bitset by iterating every run and charging for its length (`run_bitset_container_intersection_cardinality`, `roaring.c:10901`).

A compute-priced CRoaring baseline, and why it must be a second pass

Because `DEFAULT_MAX_SIZE` prices storage rather than query cost, comparing against stock CRoaring would compare against a library configured for a different objective. Every CRoaring number in this paper is therefore reported against a baseline we first re-priced ourselves, so that the comparison is against Roaring's structure under *our* objective and not against its storage objective. The re-pricing is a runtime, in-memory pass that promotes any container still held as an array past a compute threshold into a bitset (`roaring_bitmap_storm_promote_arrays`). It is a pure representation change — the set of values encoded is unchanged, and every cardinality query returns an identical answer before and after, cross-checked against the independent oracle on every pair before any timing is taken. Nothing is serialized: the promoted state exists only for the duration of the run, so the portable format and its cardinality-inferred container types are untouched.

The threshold is not `DEFAULT_MAX_SIZE`. The measured crossover at which an array–array scalar merge loses to a bitset–bitset popcount for a count-only AND is a cardinality of 64–128; the pass uses a default of 512 to leave margin for non-uniform data.

The pass must run after `run_optimize()`, not before, and lowering the insertion threshold instead is worse than doing nothing. We first tried the obvious thing — lowering the constant that `container_add` tests — and measured a 46 per cent regression on `census1881` (1496 ns to 2186 ns per pair). The cause is that CRoaring's run conversion is not confluent with container type. Deciding whether a container should become a run compares the run encoding against the size of the encoding it currently has: for an array container that is $2c+2$ bytes and scales with cardinality, but for a bitset container it is a

fixed 8192 bytes. Promoting a borderline container to a bitset *before* run conversion therefore changes which comparison that container faces, and flips containers into runs that would otherwise have stayed arrays — a worse choice for AND-cardinality cost on the weakly clustered data this paper is concerned with. Running the promotion afterwards removes the interaction: run conversion has already made its choice on its own terms, so any container still an array has already been judged not worth converting, and the only decision left is a pure array-versus-bitset compute call. This ordering dependence is a property of the library's conversion rules rather than of our pass, and it is the reason a compute-priced Roaring cannot be obtained by retuning a single constant.

Benchmark protocol and cache-residency statement

Every reported cell variant, including CRoaring, was compiled with the same compiler and flags, allocated with the same alignment, and given the same warm-up treatment before timing, so that no comparison advantages one implementation by construction. CRoaring was tuned in good faith before every comparison: `run_optimize()` and `shrink_to_fit()` were called before timing began, matching what a competent user of the library would do, and every ratio against CRoaring reported in this paper is computed against that tuned baseline, not against CRoaring's untuned default construction.

Per-cell density and strategy sweeps (Figs. 2 and 3) raced every candidate variant over an identical, pre-generated pair list in an identical order, so that the cache state each variant encountered at call time was comparable across variants; each variant's differential correctness against the oracle (below) was checked before any of its timings were accepted. A warm-up pass of **[warm-up iteration count or criterion]** preceded timed repeats. Wall-clock time was measured with a monotonic per-run timer; where a hardware cycle count is reported, it is derived from wall time and a frequency measured on the host at run time rather than assumed from a nominal clock speed, and is labelled as a derived quantity rather than a hardware performance-counter reading. On Apple silicon hosts, benchmark threads were requested onto the performance core cluster to avoid the heterogeneous core migration that otherwise mixes two different microarchitectures' timings into one nominally single-threaded measurement.

The working-set-size sweep behind the residency comparison (Fig. 3b) varied corpus footprint from **[smallest working-set size in the sweep]** to **[largest working-set size in the sweep]** per host, and cache-residency boundaries (L1, L2 and shared/last-level cache) were identified from each host's own reported cache geometry rather than assumed to be uniform across hosts (below). Every throughput number reported anywhere in this paper is qualified by the cache-residency regime it was measured in, named once here by regime (L1-resident, L2-resident, DRAM-resident) so that Results and Discussion can use the regime name as shorthand rather than restate cache sizes at each mention.

Statistics and aggregation

Two different aggregation conventions were used, for two different kinds of comparison, and neither is substituted for the other anywhere in this paper. For controlled synthetic sweeps in which every variant races the identical pair list under identical conditions (the density sweep, the asymmetric-cell strategy comparison, the working-set-size sweep), the reported cost is the *minimum* over **[repeat count for synthetic per-cell sweeps]** repeats, on the standard microbenchmarking rationale that the minimum estimates the underlying cost with scheduler and system-noise inflation removed, while the mean instead reports how noisy the run happened to be. For the real-corpus comparison (Table 1), where repeats necessarily interleave across corpora and share a host with other system activity over a longer measurement window, the reported ratio is instead the *median* over **[repeat count for real-corpus comparisons]** interleaved repeats, reported together with the observed range; no per-corpus ratio in this paper is quoted to two significant figures without that range being available alongside it.

Run-to-run variance was largest for the corpora and hosts named explicitly in Results and in the notes to Table 1; any corpus whose repeats disagreed beyond **[stated dispersion tolerance used to flag a corpus as unquotable]** was excluded from headline ranges as unquotable, and named as excluded rather than silently dropped. On **[count of corpora, of the total corpus set, where the winning cell itself changed between repeats]** corpora, the winning cell reported in Table 1 itself changed between repeats; this is disclosed here once so that Table 1 can report a single, clearly labelled cell per corpus without re-explaining the variance underlying it at every mention.

Correctness oracle and differential testing

Every cell variant is checked against an independent scalar oracle that computes intersection cardinality by direct bit-by-bit or element-by-element counting, sharing no code path with any of the variants under test, before that variant is admitted to any timed comparison; a variant that returns a wrong cardinality is not reported as fast regardless of its measured speed. Correctness is tracked at three levels: an ABI-level regression suite (**[test_storm check count]** checks against the oracle, **[test_storm failure count, expected "0"]** failures) that is compiled and linked as C to serve as the public C ABI's own regression test; a cell-level differential suite (**[test_cells check count]** checks, **[test_cells failure count, expected "0"]** failures) that exercises every implemented cell variant against the oracle across a grid of densities, structures and alignments; and a larger differential run, on the order of **[cross-architecture differential check count per host family]**, executed once per host family during

the cross-architecture measurement campaign. The public C ABI surface was verified independently with `nm`: **[count of unmangled STORM_-prefixed exported symbols]** unmangled `STORM_` symbols and **[count of defined mangled (C++) symbols, expected "0"]** defined mangled symbols, confirming that the public interface is C-linkable as declared.

The cell-level suite is driven by a fuzz grid over {row count, word count, density, within-row structure (uniform, clustered, run-based), alignment}, following the same axes used for every kernel change in this project. The regression suite's own ability to fail was itself verified rather than assumed: each of a set of known defects fixed earlier in the project was reverted individually and the suite was confirmed to fail before being re-fixed, establishing that a red result from the suite is informative and not merely a suite that always passes.

Hardware and toolchain

Measurements were taken on **[count of microarchitectures used across the whole paper, expected "four"]** microarchitectures spanning two independently hand-written SIMD kernel paths — one using ARM NEON, the other using x86-64 AVX-512 with the `VPOPCNTDQ` extension — plus a portable scalar fallback used where neither hand-written path applies. No architecture-specific SVE or SVE2 kernel exists in this project's kernel sources at the time of writing: the Arm Neoverse-class hosts used here run the generic NEON path, on hardware that is SVE2-capable but for which no SVE2-specific intrinsic code was written, and no result in this paper should be read as evidence about an SVE2 kernel. The same applies to AVX2 and SSE4.2: build options exist to gate an older, separate code path (described below) to those instruction sets, but no pairing-matrix cell was written against them.

Each host is identified by **[per-host microarchitecture name, core/cluster configuration, and nominal clock speed]** and by its cache hierarchy (**[per-host L1d, L2, and shared/last-level cache sizes]**), cross-referenced against the cache-residency regimes named above rather than restated per measurement. All code was compiled with **[compiler name and version per host]**, using **[compiler flags used, and whether native-architecture dispatch (`-march=native` or equivalent) was enabled or disabled for a given reported number]**. The CRoaring baseline used throughout is pinned at version **[CRoaring version pin, expected "v4.7.2" pending re-verification against current `master`]**, vendored as a source amalgamation within the project repository, so that the exact baseline is reproducible rather than referred to generically as "CRoaring."

Data and code availability

The benchmark harness, kernel sources, and per-corpus loader scripts described above are available in the project repository under the Apache License, Version 2.0. The pairing matrix, the selector and the tile pipeline described in this paper are not, at the time of writing, exposed through the project's public C ABI; the public ABI currently surfaces an earlier, separate code path not used for any measurement reported here. This paper reports a systematic measurement of the pairing-matrix mechanism as implemented in the project's benchmark and kernel sources, not a released, ABI-stable tool, and this is stated here as a fact about the present state of the code rather than a property that is apologized for. Public real-world corpora were obtained from their original sources and reproduced locally with the loader scripts referenced above; raw per-corpus timing data, including the repeats and dispersion referenced in Statistics and aggregation, is deposited as Supplementary material.

Supplementary Information

Supplementary Note 1: The fixed-cost headroom argument

Relocated from the Introduction, where it followed the statement that the $\Theta(m)$ bitmap kernel pays the same fixed cost regardless of skew, spending most of that cost combining zeros against zeros (main text).

That waste is not an anecdote about one kind of data; it is a property of the fixed cost itself, and its size can be settled before any kernel is written. Every alternative encoding of the same set — a sorted array of its elements, a list of its maximal runs, a fill-compressed word stream, or any of these applied to its complement — is exact and interconvertible with the bitmap, and each is sized by the set rather than by the universe. The cheapest of them is smaller than the bitmap by a factor that grows without bound as either operand moves away from density one half, so a flat cost provably leaves headroom at every density but the middle of the range, for a single operation as much as for a workload of them. Assuming nothing about the data beyond its density gives a floor on that headroom rather than an estimate of it: independently placed elements are the worst case for every encoding whose cost depends on how a set is arranged, so real structure moves those encodings further below the bitmap and never above it. What this paper optimizes is therefore not how fast a kernel runs but how much of the computation is never entered. The same analysis says where that is not possible. Around density one half no encoding is smaller than the bitmap and nothing can be skipped; there the fixed-cost kernel is the right one, and the return comes from executing it as fast as the hardware allows — which is what two decades of vectorized `AND`-and-popcount engineering already deliver[6]. Avoid the work where it can be avoided, accelerate it where it cannot: this paper contributes the first axis, and a cheap way of knowing which of the two applies.

Supplementary Note 2: Narrowing the query

Relocated from the Introduction, where it followed the statement of Roaring's fixed, per-chunk granularity (main text).

All of that leaves the caller's question untouched, and a caller willing to narrow it can be given more. If only pairs above a similarity threshold are wanted, or only operands inside a band of set sizes, then the two cardinalities alone cap the similarity a pair can reach, so most pairs are discarded before either operand is read[1,3] — knowing what is being looked for is itself a way of not doing work, and it is the cheapest one available.

Supplementary Note 3: Thresholded-mode filtering — the size bound and prefix filtering

Full derivation for the opt-in thresholded mode named third in the mechanism ladder that opens Results, and stated with its hypotheses in Methods, "What makes work avoidable" (Eq. (8)–(10)): only pairs whose Jaccard similarity or intersection size clears a threshold are reported. Two mechanisms serve that mode, and the cheaper of them touches no data whatsoever. Because $|A \cap B| \le \min(a,b)$ while $|A \cup B| \ge \max(a,b)$, a pair's similarity is capped by the ratio of its two sizes, $J(A,B) \le \min(a,b)/\max(a,b)$, so a pair whose sizes differ by more than a factor $1/t$ cannot reach the threshold and is discarded from two integers, before either operand is read. An absolute threshold behaves the same way: $|A \cap B| \ge \tau$ requires only $\min(a,b) \ge \tau$. Restricting the *inputs* to a band of sizes — ignoring the extremely rare, or keeping only those — is that same comparison applied before a pair is formed rather than after. The bound is the Swamidass–Baldi pruning result[3] together with the size (length) filter of[16], shipped over popcount-sorted fingerprint bitmaps at all-pairs scale as BitBound in chemfp[5], and layered as a popcount-based bound onto set-similarity joins by[24]; what is reported here is not the bound but where it sits relative to the rest and how it composes with them.

How much it removes is fixed by one property of the corpus, and that property is what makes it worth separating from the mechanisms above. Under the $1/i$ cardinality spectrum this paper benchmarks — cardinalities log-uniform on $[1,K]$, which is the density $\Pr(a) \propto 1/a$ exactly, so this is the mandated spectrum and not a stand-in for it — the fraction of pairs that can possibly qualify is $1-(1-\ln(1/t)/\ln K)^2$ (Methods, Eq. (9)), which agrees with a 10^5-sample simulation to three decimals. At $K = 10^4$ that leaves 14.5% of pairs at $t = \frac{1}{2}$, 4.8% at $t = 0.8$, 2.3% at $t = 0.9$ and 1.1% at $t = 0.95$, so examining every pair does 6.9× to 90.0× the work of examining only the survivors. The expression falls as K rises: the wider the spread of set sizes, the more pairs are ruled out by the ratio alone. That is the reverse of the dependence every other mechanism in this paper has. The null models of Fig. 2 are all stated at a *fixed* density and bound what a mechanism is worth against within-row arrangement; the size bound is the only one whose value is a function of the *dispersion of cardinalities across the corpus*, and it strengthens monotonically as that dispersion widens — 2.3% surviving at $K = 10^4$ against 1.1% at $K = 10^8$, both at $t = 0.9$. The two axes are different and not in competition. One limit is worth stating with it: the favourable scaling belongs to this spectrum rather than to skew in general, and under a heavier tail in which *ranked* sizes scale as $1/i$ the surviving fraction is exactly $1-t$ whatever K is (Methods, "What makes work avoidable").

It does not, however, multiply with the mechanisms below it, and the reason refines the composition rule this section has otherwise used. Deciding whether a pair exists at all is the coarsest granularity in the paper, so the rule as stated predicts a clean product with anything acting inside a pair. The product is not achieved. The survivors of the size test are the pairs with $a \approx b$, and representation choice, a zone map and the prefix filter are each cheapest precisely when the two sizes are *dissimilar*

— so the test passes on the pairs the rest serve worst, and the composed cost exceeds the product of the two savings by the factor that conditioning raises a survivor's mean cost, $\ln K/2$ in the limit of a high threshold (Methods, Eq. (10)). At $K = 10^4$, $t = 0.9$ and $m = 10^6$ the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition costs 0.151%. Replaying the same composition with the size test decorrelated from cardinality, at an identical survival rate, restores the product to within 0.6%, which identifies the shared statistic rather than the shared granularity as the cause. The composition remains firmly worth taking — it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does 660× its work — so what this refutes is the product, not the composition. Mechanisms compose multiplicatively when they act on different waste *and* read different operand statistics; acting at different granularities is necessary and not sufficient. These are analytic values over the model above, and like the rest of this note they count work rather than time.

The thresholded mode changes the contract rather than the kernel: only pairs above a similarity threshold are reported, and the second mechanism that exploits that is prefix filtering rather than any representation choice. It admits the same analysis, and it reduces further than the others do. Because a qualifying pair must share an element between its prefixes, the entire behaviour of the filter is governed by one dimensionless quantity — the expected number of shared prefix elements per pair, $y = p^2/m$ — and the fraction of pairs surviving is $1 - e^{-y}$ regardless of density, threshold or universe size separately (Supplementary Fig. S1). Below $y = 1$ pruning improves quadratically as prefixes shorten; above it prefixes collide by default and the index earns nothing. That single variable is what a gate for this mode should test, and it is cheap to compute from the metadata already held.

Supplementary Note 4: Derivation of the zone-map cost model and its crossings

Relocated from Results, "The cost of intersection is U-shaped in density," which points here for the full derivation behind Fig. 2b–d: the null-model floor, the four counting properties of a zone map (why it has a floor, why the leverage is large, why the advantage is flat then erodes quadratically, and why it can be a liability), the crossing densities, and the work-versus-time distinction.

Every panel of Fig. 2 is the same argument applied to a different mechanism, and it is worth stating that argument once. Each mechanism — choosing a representation, consulting a zone map, composing the two, and in the thresholded mode the prefix filter of Supplementary Fig. S1 — is a bet that the data deviates from randomness, and each comes with a closed-form null model saying what the bet is worth if it does not. Fixing the density and placing elements independently is the most pessimistic assumption available about *arrangement*, and it is pessimistic in the same direction for all of them: independent placement maximises the number of maximal runs a set has, maximises the fraction of bins it occupies, and maximises the chance that two prefixes collide. The null model is therefore a floor and not a prediction. Real structure can only move each mechanism further into profit, and the size of that movement is exactly what the mechanism converts into speed.

Two things a fixed design cannot do follow immediately, and they are the reason the selector is built the way it is. A representation committed to at ingest cannot exploit the deviation, because it commits before the deviation is knowable. A selector reading density alone cannot exploit it either, because density is the very statistic the null model is built from: at a fixed density d with $k = dm$ elements the expected number of maximal runs is $k(m-k+1)/m$, but the actual number ranges over every integer from one to $\min(k, m-k+1)$, so no single function of d can track it. The deviation is also *one-sided* — a real row can be arbitrarily cheaper than a random row of the same density and at most about twice as expensive (Methods, "What makes work avoidable") — which is what makes it a resource rather than a source of variance. Both readings point the same way: the statistic a selector consults has to see arrangement, or be a direct measurement of the kernels themselves.

Four properties of the summary in Fig. 2b–c follow from counting alone, and they are worth stating plainly because together they say what a summary can and cannot buy.

Why there is a floor, and where it is. A zone map is a bitmap of bitmaps: one bit per bin of W_z positions, so m/W_z bits against the row's m. Reading it therefore costs $1/W_z$ of reading the row, and no pairing that consults a summary can cost less than reading that summary — 0.195% of a bitmap at $W_z = 512$. That is the floor, and it is a property of the bin width alone, not of the data.

Why the leverage is so large. The exchange rate is the same W_z: one summary bit decides the fate of W_z data bits, so a bin proved empty returns W_z bits of work for the one bit spent asking. This is why a coarser bin buys a deeper floor — and, symmetrically, why it stops paying at a lower density, since a wider bin is harder to leave empty.

Why the advantage is flat across the sparse range and then erodes quadratically. A bin survives only if *both* operands occupy it, so the surviving fraction goes as the product of the two occupancies. For clustered operands, where a run longer than a bin occupies exactly a d fraction of bins, that product is d^2, and the cost is $1/W_z + d^2\kappa$ with κ the cost of a bin's contents. The two terms are equal when $d^2\kappa = 1/W_z$; with κ the cost of one run in a bin, $\kappa = w/W_z$, this reduces to $d = 1/\sqrt{w} = 1/8$ — independent of the bin width. Below that density the summary term dominates and the advantage is constant; above it the surviving-bin term dominates and the advantage decays as d^2. Changing W_z moves the floor but not the knee.

Why a summary can be a liability, and exactly when. The floor cuts both ways, and the reason is structural rather than a

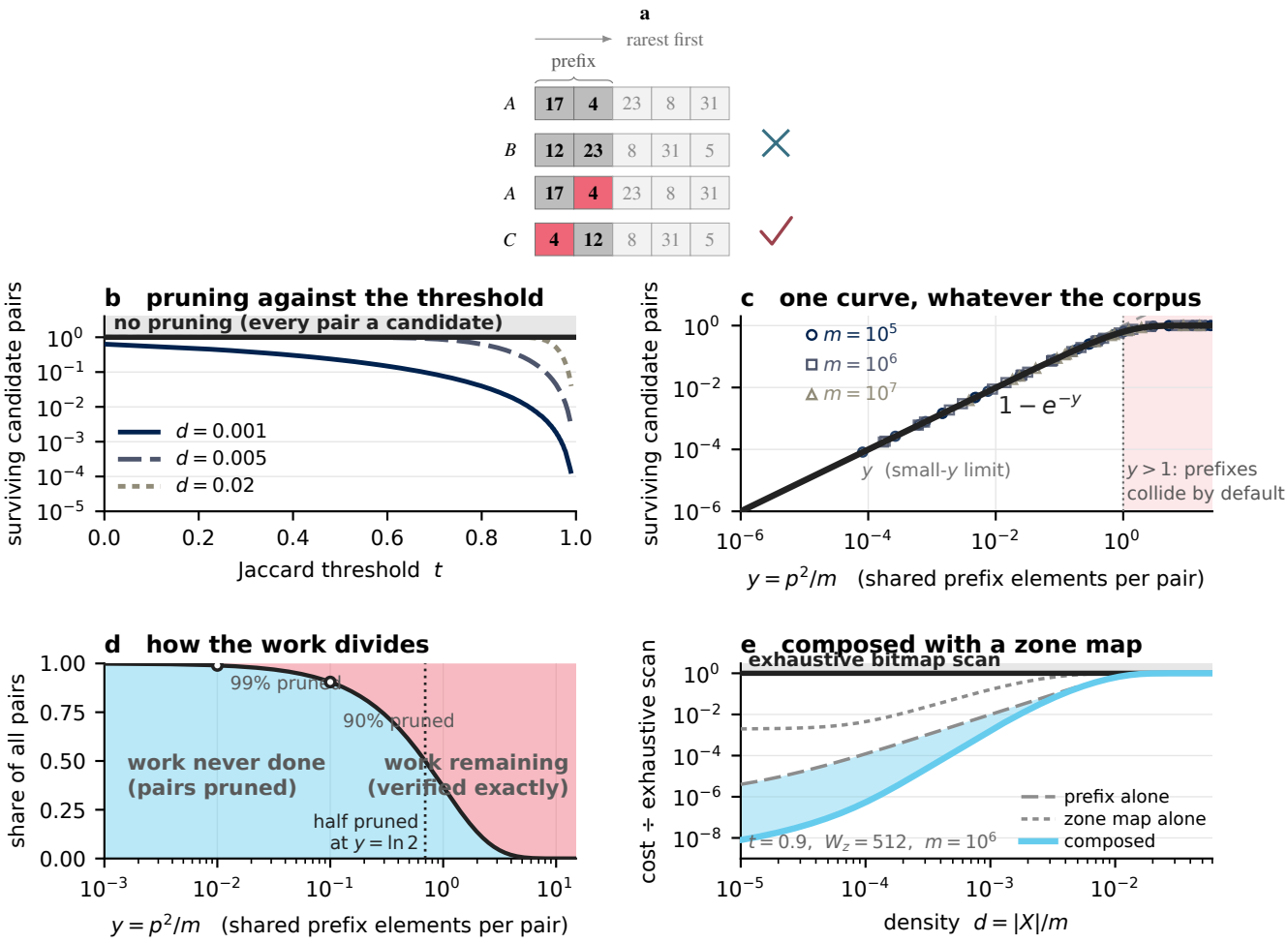

Figure S1. Prefix filtering, analysed on the same terms — and reducible to a single variable. Prefix filtering serves the opt-in thresholded mode, in which only pairs of Jaccard similarity at least t are reported. With elements in a global order and prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$[2], following the prefix-filtering principle of[25], any qualifying pair must share an element between its prefixes, so indexing prefixes alone answers the thresholded question exactly — and unlike the mechanisms of Fig. 2c–d it is candidate *generation*: the quadratic pair set is never enumerated. Reading key as in Fig. 2; an ordered series takes a sequential ramp with its own marker or dash pattern. **a**, How the rule works. Elements are laid out rarest first, hence the non-monotonic labels, and each set's prefix (saturated) is its rarest $p(X)$ elements. Top: the prefixes share no element, so the pair is discarded without computing any intersection. Bottom: one element appears in both, marked in red because it is what commits the pair to exact verification. **b**, Fraction of pairs surviving as candidates against the threshold, at universe 10^6. Prefix length falls as t rises, so the filter that removes almost everything at a high threshold removes almost nothing at a low one, and denser operands need a higher threshold to prune as hard. **c**, All of that collapses onto one curve. For prefixes of length p over a universe of m the expected number of shared prefix elements per pair is $y = p^2/m$, and the surviving fraction is exactly $1 - e^{-y}$ for scattered operands; markers are (d, t, m) combinations drawn at random, and lie on the curve by construction rather than by fit. For $y \ll 1$ the surviving fraction is y itself; for $y > 1$ prefixes collide by default and the index is pure overhead. So y — not t, and not density — is what a selector should test. **d**, The same as a division of labour: pairs never examined against pairs surviving to exact verification. The two are equal at $y = \ln 2$. **e**, Prefix filtering composed with a zone map. Unlike the two mechanisms of Fig. 2c these *do* compose — their savings multiply, to within **[a factor of 1.25 at the sparsest end, exactly elsewhere; analytic]** — because they act on different waste and read different statistics. At $d = 10^{-3}$, $t = 0.9$ and $W_z = 512$ the composition costs **[$\approx 0.17\%$, analytic]** of an exhaustive bitmap scan, against 1.0% for prefix filtering alone and 16% for a zone map alone. Analytic throughout; no measured data, and like Fig. 2 it models work rather than time. Real corpora deviate in the same direction as elsewhere: frequency-ordered prefixes concentrate rare elements, so real prefixes collide less than scattered ones of the same length and the true curve lies below this one.

property of any one representation: a summary is indexed by the universe, while every compressed representation is indexed by the data. The zone map costs $1/W_z$ of a bitmap whatever the operands contain, because its size is set by m. The cost of S, R and W is set instead by a quantity that shrinks with the set — cardinality, run count, or literal-and-fill count — and each of those falls without limit as the operands sparsify. A fixed cost and a vanishing one must cross. Below that crossing, consulting a summary costs more than the entire computation it was meant to shorten, and a zone map is worse than not having one.

Where the crossing falls depends on which representation is being displaced, and one case fixes the scale. A representation costing one word per element costs wd of a bitmap, so it meets the summary's $1/W_z$ at $d = 1/(wW_z) = 3.1 \times 10^{-5}$ for $w = 64$ and $W_z = 512$ (Methods, Eq. (7)). At $d = 10^{-6}$ that representation costs 6.4×10^{-5} of a bitmap while the summary alone costs 1.95×10^{-3}, so consulting it does $31\times$ the work of simply enumerating the operands. A run- or fill-based representation crosses at its own density, wherever its run or word count falls below m/W_z, but that a crossing exists at all is generic.

This makes the gating two-sided, and for two unrelated reasons. A summary must be declined when the operands are *too dense*, because every bin is occupied and it proves nothing; and equally when they are *too sparse*, because it is then the largest object in the computation. It pays only between those two bounds. The same fact appears in the storage measurements as universe-proportional selector metadata dwarfing the data it describes; the time and space statements of it are one phenomenon, and a guard on one is a guard on the other.

The practical reading is that the sparse regime is not a narrow operating point. In the clustered idealization — the favourable edge of the band, where a run spans a whole bin — the advantage is the same at $d = 10^{-6}$ as at $d = 10^{-2}$: five orders of magnitude of density over which a fixed-cost kernel does $512\times$ the work of one that consults a summary first. It is only above $d = 1/8$ that the margin narrows at all, and even at $d \to 1$ a bitmap still does $7.9\times$ the work.

All of the quantities above are counts of 64-bit words touched, not times, and the distinction is load-bearing rather than pedantic. A work model prices a summary that removes nothing at its own size, $1/W_z$ of a bitmap, and that is a correct statement about words. It is not a statement about runtime: a filtered scan gives up sequential access and hardware prefetch and pays an indirection and a branch per bin, and the measured penalty for consulting a summary on operands it cannot help is far larger than these panels charge — **[worst-case speedup, filter applied unconditionally, which the work model does not predict]** against the $1.002\times$ overhead the work model allows, matching the always-on-versus-gated contrast reported directly in Results, "Selection must be hoisted, and measured rather than modelled" (Fig. 4c). That divergence is not a defect in the derivation; it is the reason the gate is decided by measurement rather than by the model, and the per-bin and per-word constants needed to convert these curves into time are given in Methods.

Supplementary Note 5: What repeats across a batch, and how each mechanism amortises it

Relocated from Results, "Selection must be hoisted, and measured rather than modelled," which points here for the four quantities that repeat when a batch of pairs is run naively and how each is amortised instead.

The contract is unchanged and the answer is identical, but batch amortisation requires many operations rather than one, because what it removes is not work but *repetition*. Four things repeat when a batch is run naively, and each can be paid for once instead. Row metadata — cardinality, run count, occupancy summary — is part of the representation, built at ingest and amortised over every operation the set ever participates in rather than over any particular batch; this is the same assumption Roaring's per-chunk container choice already makes, and it is stated here once for the whole paper. A selection decision can be hoisted to a tile, so one decision serves a block of pairs instead of one pair. Probe-and-commit pays to time a handful of operations and reuses the verdict for the rest. And operand data loaded once can be paired against many, so its memory traffic is amortised rather than repeated. In each case the same thing happens: the decision and the metadata are amortised, and only the kernel work stays per operation (Supplementary Fig. S2b). This is a regime rather than the setting. It applies wherever there are enough operations to share a decision — which is a property of a workload of any size, not an argument that anything must be computed exhaustively — and it is the regime in which the selection-cost measurements of the main text were taken. The single-operation regime is reasoned instead from the cost of a decision read from $O(1)$ metadata against the size of the operation it governs, and the asymmetry in evidence between the two is stated here rather than left to be inferred.

Two mechanisms of that regime are drawn together in Supplementary Fig. S2: an overlap pre-filter that settles most of a pair set from summaries alone, and the way a decision's cost is governed by the size of the operation it is made about.

Supplementary Note 6: Corpus provenance and inventory

Supplementary Table S1 gives full provenance and per-corpus statistics for every corpus referenced anywhere in this paper, including the corpora scored only against an all-bitmap baseline and never against CRoaring, which Table 1 summarises in a single row. It also records, for the two genomic corpora, which axis is treated as the universe under each storage layout — the distinction the main text draws in Results, "Where the method does not pay."

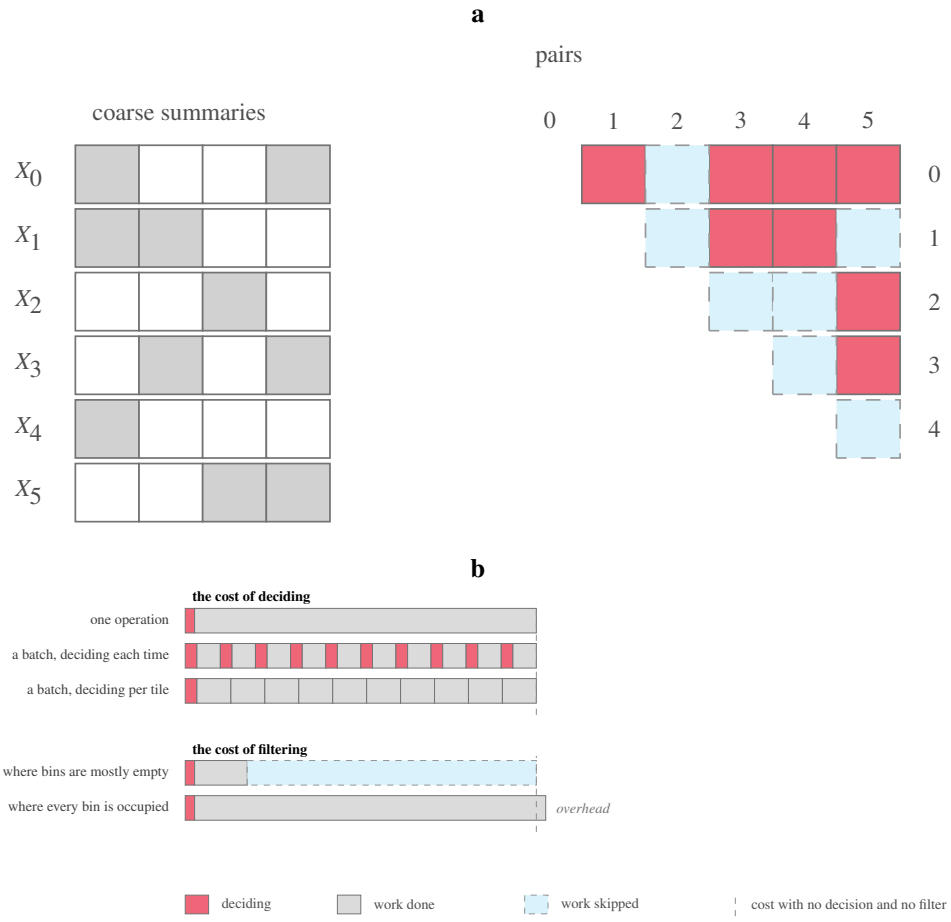

Figure S2. The batch regime: rejecting pairs wholesale, and paying for a decision once. Both mechanisms here need many operations rather than one, and what they remove is repetition. **a**, An overlap pre-filter across a set of operands. The same occupancy idea as a zone map (Fig. 2a) but at a coarser, chosen width and computed once per set, then ANDed pairwise: shaded pair cells may overlap and are passed to a kernel, dashed cells are provably disjoint and are answered without touching data. One pass over the summaries settles most of the pair matrix. Note the distinction from prefix filtering (Supplementary Fig. S1), which is candidate *generation* and never enumerates the pair set at all. **b**, Why the cost of a decision depends on what it governs. Each bar spends left-to-right through one unit of useful work; the dashed rule marks that work with neither a decision nor a filter. *Deciding*: a single operation absorbs the decision easily, since it is read from $O(1)$ metadata and the operation it governs is large; the same decision repeated for every operation of a batch is a large fraction of the total, because each operation there is small; and hoisting it to a tile — deciding once, then committing — restores the single-operation picture. *Filtering*: where bins are mostly empty a summary converts most of the work into skipped work; where every bin is occupied it finds nothing to skip and its cost carries the bar past the rule. The cost of deciding therefore scales inversely with how large the operation being decided about is, which is why the two regimes need different policies and why every mechanism is gated rather than always on. Schematic; no measured data.

Table S1. Corpora, provenance and statistics. Every corpus referenced anywhere in this paper, in four provenance groups used to assemble the benchmark set: **G1**, the twelve `real-roaring-datasets` files CRoaring's own harness runs by default[8,9]; **G2**, five SNAP social/web graphs (Stanford Large Network Dataset Collection) scored by neighbourhood-intersection density; **G3**, two genomic corpora present for scoping rather than for the headline, included so a reader can judge whether the benchmark set was chosen favourably; **G4**, KONECT graphs, UShER SARS-CoV-2, gnomAD and the `msprime` coalescent-simulation series, added after the original survey. Density and mean $|X_i|$ are corpus properties from the data loaders, not timing results. For the two G3 corpora, *universe* and *rows* swap roles between layouts: `chr20.bin` is variant-major (universe = 5,008 haplotypes, one row per variant site); `chr20_hap.bin` is haplotype-major (universe ≈ 1,048,576 sites, one row per haplotype) — the layout dependence discussed in the main text (Results, "Where the method does not pay"). —, not reported by the source. [All values provisional, pending the final measurement campaign.]

Corpus	Group	Rows/sets	Universe m	Mean $\|X_i\|$	Density	Origin / notes
`census1881`	G1	200	4,277,806	5,019	1.2e-3	1881 Canadian census, categorical attributes
`census1881_srt`	G1	200	4,277,735	3,404	8.0e-4	as above, rows sorted
`census-income`	G1	200	199,523	34,610	1.7e-1	US census income (UCI "Adult"-family), attributes
`census-income_srt`	G1	200	199,523	30,464	1.5e-1	as above, rows sorted
`weather_sept_85`	G1	200	1,015,367	64,353	6.3e-2	September 1985 weather observations, binned attributes
`weather_sept_85_srt`	G1	200	1,015,367	80,540	7.9e-2	as above, rows sorted
`wikileaks-noquotes`	G1	200	1,353,179	1,377	1.0e-3	WikiLeaks cable corpus, term/attribute postings
`wikileaks-noquotes_srt`	G1	200	1,353,133	1,440	1.1e-3	as above, rows sorted
`uscensus2000`	G1	200	36,974,578	30	8.1e-7	US Census 2000; sparsest and largest-universe G1 corpus
`dimension_003`	G1	15,482	3,866,847	91	2.4e-5	Druid-derived dimension column
`dimension_008`	G1	5,240	3,866,845	347	9.0e-5	Druid-derived dimension column
`dimension_033`	G1	173	3,866,847	22,352	5.8e-3	Druid-derived dimension column
`as-skitter`	G2	1,696,415	1,696,415	[n]	9.1e-6	CAIDA skitter internet AS topology, 2005; undirected, symmetrized
`wiki-Talk`	G2	2,394,385	2,394,385	[n]	2.2e-5	Wikipedia talk-page edits; directed; only 67,492 vertices have out-degree ≥ 2
`soc-Pokec`	G2	1,632,803	1,632,804	[n]	1.6e-5	Pokec social network (Slovakia); directed; lowest skew of the graphs – top 1% of sets hold only 8.8% of elements
`com-LiveJournal`	G2	3,997,962	4,036,538	[n]	5.1e-6	LiveJournal friendship, ground-truth communities; undirected, symmetrized
`com-Orkut`	G2	3,072,441	3,072,627	[n]	2.7e-5	Orkut social network; undirected, symmetrized; largest input in this set
`chr20.bin`	G3 (scoping)	[n]	5,008	[n]	3.5e-2	1/i site-frequency spectrum, 43.6% singletons; variant-major, universe 626 bytes, L1-resident
`chr20_hap.bin`	G3 (scoping)	[n]	≈1,048,576	[n]	3.1e-2	haplotype-major; large universe but 3.1% dense – nothing beats all-bitmap (0.93×) at this density and layout
`dbpedia-link`	G4	4,384,102	18,268,993	29.8	1.63e-6	KONECT
`wikipedia_link_en`	G4	5,978,043	11,206,012	46.4	4.15e-6	KONECT
`livejournal-groupmemberships`	G4	2,197,915	7,489,074	48.7	6.50e-6	KONECT, bipartite; universe inflated ∼2.3× (single id space for both columns; true member domain is 3,201,203)
`usher_sarscov2`	G4	29,411	8,451,771	15,769.3	1.87e-3	UCSC UShER SARS-CoV-2; mean dominated by near-universal variants (max 99.7% carriers), median only 404
`enwiki-categorylinks`	G4	2,165,205	129,698,523	102.3	7.89e-7	Wikimedia dumps; largest universe, lowest density in the full set
`gnomad_chr21_exomes_af1e-3`	G4	625,656	1,461,894	28.2	1.93e-5	gnomAD v4.1
`msprime_10k`	G4 (sim)	≈42k	2e4	1,887	9.4e-2	msprime coalescent simulation; out of target regime
`msprime_100k`	G4 (sim)	≈51k	2e5	15,720	7.9e-2	as above; out of target regime
`msprime_1M`	G4 (sim)	≈15.6k	2e6	136,288	6.8e-2	as above; out of target regime

Supplementary Note 7: Selector-metadata overhead and storage cost

Storage is a reported trade-off rather than an objective this paper optimizes for. The two tables below separate the two costs that the main text keeps apart. Supplementary Table S2 gives the metadata the selector itself needs, per row and against the data it

Table S2. Selector metadata is universe-proportional, and that is a defect. The rank index and zone map are constructed unconditionally, and both scale with universe, not cardinality. Meta B/row and data B/row are bytes of selector metadata and bytes of the chosen data representation, per row; ratio is meta:data. On `enwiki-categorylinks` that is 4 MB of selector metadata attached to 155 bytes of data – a 26,312× overhead. The complement representation already carries a guard, built only when $2n >$ universe; rank and occupancy carry none. Rows on which metadata exceeds data by more than $10^3\times$ are bold. This is a known, unfixed property of the current implementation rather than a design choice, and it is why bitmap-bearing measurements elsewhere in this paper are restricted to small row counts on the large-universe corpora. [All values provisional, pending the final measurement campaign.]

Corpus	Universe m	Meta B/row	Data B/row	Ratio (meta:data, ×)
`census-income`	199,523	6,335	13,344	0.5
`weather_sept_85`	1,015,367	32,031	54,497	0.6
`census1881`	4,277,806	134,784	18,774	7.2
`as-skitter`	1,696,415	53,479	49	**1,083**
`dbpedia-link`	18,268,993	575,415	113	**5,051**
`enwiki-categorylinks`	129,698,523	4,084,799	155	**26,312**

Table S3. Storage cost, bits per value, data and metadata separated. Reported the way the Roaring papers report it[23], so corpora of different cardinality are comparable. Metadata is excluded from every column here: the zone map and rank index are what the selector needs to *choose* a representation, not a representation themselves, and folding them into a data column would hide the cost of the mechanism this paper adds (Supplementary Table S2). **Best** is the minimum of the four representation columns, per corpus, and is set in bold in every row for that reason. Bitmap costs are given in scientific notation throughout, one notation down the column. On dense corpora the numbers are Roaring-comparable (3–7 bits/value); on sparse corpora a dense bitmap costs 10^3–10^6 bits/value, the storage statement of the same fact the timing tables make: a bitmap over a large sparse universe is not a compression scheme, it is a liability. [All values provisional, pending the final measurement campaign.]

Corpus	Universe m	Bits per value				
		Bitmap	Array	Runs	EWAH	**Best**
`census-income`	199,523	5.77e0	32.00	20.73	3.86	**3.08**
`msprime_100k`	200,000	1.26e1	32.00	31.88	5.29	**4.44**
`weather_sept_85`	1,015,367	1.58e1	32.00	30.06	7.87	**6.77**
`usher_sarscov2`	8,451,771	1.08e3	32.00	2.04	4.06	**2.02**
`wikileaks-noquotes`	1,353,179	9.83e2	32.00	11.36	19.46	**10.64**
`census1881`	4,277,806	8.52e2	32.00	58.86	43.79	**29.92**
`as-skitter`	1,696,415	1.26e5	32.00	47.13	77.72	**29.26**
`wikipedia_link_en`	11,206,012	2.68e5	32.00	46.21	72.90	**28.27**
`dbpedia-link`	18,268,993	6.38e5	32.00	56.38	101.63	**31.84**
`uscensus2000`	36,974,578	1.24e6	32.00	57.78	91.89	**31.98**
`enwiki-categorylinks`	129,698,523	3.33e6	32.00	62.72	113.23	**31.85**

describes; it is the evidence behind the third boundary named in Results, "Where the method does not pay," and it shows that the rank index and zone map are sized by the universe rather than by the row. Supplementary Table S3 gives the cost of the chosen data representation alone, with that metadata excluded, so the two are never conflated.

Supplementary Note 8: Formal treatment of work avoidance — full proofs

Methods ("What makes work avoidable") states each result of this development as an equation with its hypotheses and a pointer here. This note supplies the proof, the worked numerical checks and the secondary results that support but do not themselves state a numbered result in Methods. Notation follows Methods throughout: $A, B \subseteq [0, m)$, $a = |A|$, $b = |B|$, $d_A = a/m$, $d_B = b/m$, word width w, effective sparsity $s_X = \min(d_X, 1 - d_X)$, $s = \min(s_A, s_B)$. For a representation ρ and a set X, $\kappa_\rho(X)$ is the number of stored items a kernel must touch to traverse X in that representation — $\lceil m/w \rceil$ words for B, $|X|$ for S, the run count $r(X)$ for R, and literal-word count plus fill-group count $\ell(X) + g(X)$ for W. All costs below are counts of stored items or of elementary operations, never a time; the constants that convert one to the other are measured, not derived, and are given in Methods under "Benchmark protocol and cache-residency statement".

Proof of Eq. (1) (the Fréchet interval) and Eq. (2) (its width). Upper bound: $A \cap B \subseteq A$ and $A \cap B \subseteq B$, so $|A \cap B| \leq \min(a, b)$. Lower bound: $|A \cup B| = a + b - |A \cap B| \leq m$, so $|A \cap B| \geq a + b - m$; together with $|A \cap B| \geq 0$ this gives the stated interval. Every integer t in the range is attained: take $A = [0, a)$ and $B = [a - t, a - t + b)$, so that $A \cap B = [a - t, a)$ has size t. The

construction is legal because $t \le a$ gives $a - t \ge 0$, $t \ge a + b - m$ gives $a - t + b \le m$, and $t \le b$ makes $[a-t, a) \subseteq B$.

For the width, $\min(a,b) - \max(0, a+b-m) = \min(a,b) + \min(0, m-a-b) = \min\big(\min(a,b), \min(a,b)+m-a-b\big)$, and $\min(a,b)+m-a-b = m - \max(a,b)$, so the width is $\min\big(\min(a,b), m-\max(a,b)\big) = \min(a,b,m-a,m-b)$. Grouping the four terms in pairs, $\min\big(\min(a,m-a), \min(b,m-b)\big) = \min(ms_A, ms_B) = ms$. The number of feasible answers is $ms+1$, so a cardinality-only view of a pair leaves $\log_2(ms+1)$ bits of the answer unresolved. The four terms $a, b, m-a, m-b$ are exactly the sizes of the four lists A, B, A^c, B^c one could enumerate, so the residual uncertainty is the size of the smallest of them — not a coincidence, since Eq. (3)'s proof below shows the smallest of those four lists is precisely what an exact algorithm enumerates to answer the query.

The pigeonhole floor. The lower bound of Eq. (1) lifts off zero exactly when $d_A + d_B > 1$, giving $|A \cap B| \ge (d_A + d_B - 1)m$; equivalently, by inclusion–exclusion on complements, $|A \cap B| = a + b - m + |A^c \cap B^c|$, so the floor is the case $A^c \cap B^c = \varnothing$: the complements can be at most disjoint. At $d_A = d_B = 0.51$ the floor is $(2 \cdot 0.51 - 1)m = 0.02m$ — the factor is two, so 51% density forces 2% of the universe to overlap, not 1%; as a fraction of either operand it is $2 - 1/d \approx 3.92\%$. Above $d = \frac{1}{2}$ the floor rises at $2m$ per unit density while the ceiling $\min(a,b) = dm$ rises at m, so the width $m(1-d)$ shrinks at the constant rate m per unit density — consistent with $m(1-d) = ms$ for $d \ge \frac{1}{2}$.

Proof of Eq. (3) (the headroom). $\kappa(\mathsf{B}) = \lceil m/w \rceil$ for every X by definition: a dense bitmap stores m bits whatever they contain, so $\mathsf{B}\times\mathsf{B}$ performs $\lceil m/w \rceil$ word AND-plus-popcount operations for every pair independently of a, b, of structure and of the answer. This is not a defect of any implementation; B is the only member of the matrix whose descriptor length is a function of the universe alone.

Suppose each operand X is available as a bitmap *and* as a sorted array of whichever of X, X^c is the smaller — $\Theta(ms_X)$ of extra space, decided from a, b, m in $O(1)$. No rank index is required for what follows; that enters separately, below. Without a stored complement array, enumerating X^c from the bitmap costs $\Theta(m/w + (m-a))$, whose first term is already $\kappa(\mathsf{B})$ — so the dense arm of the argument genuinely requires the complement to be materialised, not derived on the fly. Then $|A \cap B|$ is computed in $O(ms)$ constant-time membership probes, by four cases, one per term of $\min(a, b, m-a, m-b)$: if $U = a$, enumerate A's a positions and probe each into B's bitmap (a probes); if $U = b$, symmetric; if $U = m - a$, enumerate A^c and recover $|A \cap B| = b - |A^c \cap B|$ ($m - a$ probes); if $U = m - b$, symmetric via $|A \cap B| = a - |A \cap B^c|$. Each case is $O(1)$ arithmetic plus U probes. Two of the four cases require the complement view, which is why C is part of the design and not an afterthought for the dense tail: without it the achievable cost is $\min(a,b)$, which is $\Theta(m)$ at high density, and the dense arm of the U does not exist.

Comparing descriptor items touched, $\kappa(\mathsf{B})/U = \lceil m/w \rceil / (ms)$. Taking $w \mid m$ (true of the implementation, which pads to a whole number of words) this is exactly $1/(ws)$; otherwise read it as $\ge$ with an additive w/m term (relative error at $m = 1000, w = 64$ is 2.4%, at $m = 65$ it is 97%; nothing downstream changes). The ratio exceeds one whenever $s < 1/w$ and is unbounded as $s \to 0$. What this does and does not assert: it asserts that a specific, exact, correct algorithm exists whose cost is ms items where $\mathsf{B}\times\mathsf{B}$ spends m/w items, and that the ratio between the two achieved costs diverges. It does *not* assert that ms is a lower bound on the work any algorithm must do ("Why the interval width is not a lower bound on work", below); both quantities compared are achieved, neither is a floor.

The crossover heuristic. Converting to time needs two measured constants: the cost of a random probe into a bitmap, c_p, and the cost of a vectorised AND-plus-popcount word, c_w/V. Equating the two costs gives the crossover $s^* = c_w/(c_p w V)$ — an explicitly constant-factor, not asymptotic, argument, and the only quantitative prediction of this kind in this note.

Proof of Eq. (4) (the rank-index bridge). This is the only result here that converts a descriptor *length* into an *operation count*; every other result in this note is a fact about storage. Let A be a bitmap carrying an $O(1)$ rank index, $\mathrm{rank}_A(i) = |A \cap [0, i)|$, and B a run container with maximal runs $\{[u_i, v_i)\}$, $i = 1, \ldots, r$. The runs partition B, so $A \cap B$ is the disjoint union of $A \cap [u_i, v_i)$, and $|A \cap [u_i, v_i)| = \mathrm{rank}_A(v_i) - \mathrm{rank}_A(u_i)$ by the definition of rank. Summing over the r runs gives Eq. (4), computed in exactly $2r$ rank lookups and $2r - 1$ additions — independent of m, of $|A|$, of $|B|$ and of the run lengths $v_i - u_i$: a run of any length is priced as a run of length one, because the kernel never looks inside it. Symmetrically, $\mathsf{R}\times\mathsf{R}$ by simultaneous merge of two sorted run lists costs $O(r_A + r_B)$ with no rank index needed, since the merge never inspects a position inside a run. This is what makes a run count a cost parameter and not merely a storage parameter; it converts descriptor length into operation count, not into time, since per-operation constants still differ across cells.

Proofs for Eq. (5) (expected descriptor lengths under a null model). Model: each position of $[0, m)$ is set independently with probability d (Bernoulli), and $w \mid m$ so every word carries w live bits. The B and S rows are exact and not asymptotic statements at all: $\mathbb{E}\,\kappa(\mathsf{B}) = \lceil m/w \rceil$ trivially, and $\mathbb{E}\,\kappa(\mathsf{S}) = md$ by linearity of expectation over the m indicator variables. For $\mathsf{C}\circ\rho$, substitute $1 - d$ for d, since complementation is an involution on density.

Run count. A maximal run of ones begins at position i iff $x_i = 1$ and ($i = 0$ or $x_{i-1} = 0$). Summing the indicator expectations over $i = 0, \ldots, m-1$ gives the exact Bernoulli $\mathbb{E}[r] = d + (m-1)d(1-d)$; under the fixed-cardinality model (uniform random k-subset) the same argument gives the exact $\mathbb{E}[r] = k(m-k+1)/m$. Both equal $md(1-d)(1 + O(1/m))$, whose leading-order

form in $s = \min(d, 1-d)$ is $ms(1-s)$, the form used in Eq. (5). The sandwich is exact, not merely leading-order: writing $p_0 = (1-d)^w$ is not needed here, but the two exact forms bound each other by

$$\tfrac{1}{2}ms \;\le\; \mathbb{E}\,\kappa(\mathsf{R}) \;\le\; ms+1, \tag{S1}$$

since fixed-cardinality $\mathbb{E}[r] = ms(1-s)+d$ and Bernoulli $\mathbb{E}[r] = ms(1-s)+d^2$ both exceed ms marginally at the dense corner ($m = 64, k = 57$ gives $\mathbb{E}[r] = 7.125$ against $ms = 7$; the worst ratio $\mathbb{E}[r]/(ms)$ tends to 2). This corrects an earlier form of the sandwich that omitted the $+1$ and is violated by 38 cases near $d \to 1$ without it.

Fill-compressed stream. Let $p_0 = (1-d)^w$, $p_1 = d^w$ be the probabilities that a w-bit word is all-zero or all-one, and $n = m/w$. Words are independent because they cover disjoint bit ranges (this is where $w \mid m$ is used). Dirty (literal) words: $\mathbb{E}[\ell] = n(1-p_0-p_1)$. Fill groups: a maximal block of consecutive all-v words begins at word j iff word j is all-v and ($j = 0$ or word $j-1$ is not all-v), so $\mathbb{E}[g_v] = p_v + (n-1)p_v(1-p_v)$ exactly. Summing over $v \in \{0,1\}$,

$$\mathbb{E}[\ell+g] \;=\; n(1-p_0-p_1)+p_0+(n-1)p_0(1-p_0)+p_1+(n-1)p_1(1-p_1) \;=\; n-(n-1)(p_0^2+p_1^2), \tag{S2}$$

which to leading order in n is $n[1-p_0^2-p_1^2]$, matching Eq. (5)'s form $(m/w)[1-(1-s)^{2w}-s^{2w}]$ (the exact form exceeds the leading-order one by at most $p_0^2+p_1^2 \le 2$ words, immaterial at any n of interest). The encoding must compress both all-zero *and* all-one fills for the symmetry claims below to hold; an encoder compressing only zero fills is not complement-symmetric. Sparse limit. $(1/w)[1-(1-s)^{2w}-s^{2w}] \to 2s$ as $s \to 0$, so under the null model W costs about *twice* S in the sparse tail, not the same — the reason W never appears on the cost envelope in either tail.

Complement symmetry (Proposition 9, supporting the paragraph "the deviation, which is the contribution"). $\kappa(\mathsf{S})(X^c) = m - \kappa(\mathsf{S})(X)$: S is *not* complement-symmetric, which is why the C strategy exists. $\kappa(\mathsf{B})(X^c) = \kappa(\mathsf{B})(X)$ trivially. R is complement-symmetric by construction, because runs of ones and runs of zeros alternate along $[0,m)$, so $|r(X^c) - r(X)| \le 1$; the sign is not a two-sided uncertainty for a given set, it is determined by boundary membership, exactly $r(X^c) = r(X) - 1 + \mathbb{1}[0 \notin X] + \mathbb{1}[m-1 \notin X]$:

$0 \in X$	$m-1 \in X$	$r(X^c) - r(X)$
no	no	$+1$
no	yes	0
yes	no	0
yes	yes	-1

Degenerate cases obey it: $X = \varnothing$ gives $r = 0$, $r(X^c) = 1$; $X = [0,m)$ gives $r = 1$, $r(X^c) = 0$. This is why $\mathsf{R}\times\mathsf{R}$ wins the dense tail with no complement machinery: R's descriptor length is nearly invariant under complementation, while S's is not, which is exactly why the array-based cells need an explicit complement view and the run-based ones do not. $\kappa(\mathsf{W})(X^c) = \kappa(\mathsf{W})(X)$ *exactly, provided* $w \mid m$: complementing every word maps all-zero words to all-one words and back and maps dirty words to dirty words, leaving the literal/fill partition untouched. This fails when $w \nmid m$: a zero-padded tail word that is all-ones over the live bits is not all-ones over the padded word, so complementation moves it between the literal and fill classes.

Proof of the U-shape as a theorem of the null model (supporting "what randomness would predict"). Each leading-order entry of Eq. (5) is, as a function of s, either constant (B) or strictly increasing on $[0, \frac{1}{2}]$: best-of-$\mathsf{S}/\mathsf{C}\circ\mathsf{S}$ is ms, increasing; R is $ms(1-s)$, and $\mathrm{d}/\mathrm{d}s[s(1-s)] = 1-2s > 0$ for $s < \frac{1}{2}$; W is $(m/w)[1-(1-s)^{2w}-s^{2w}]$, with derivative $(m/w)\cdot 2w[(1-s)^{2w-1}-s^{2w-1}] > 0$ for $s < \frac{1}{2}$. Each is symmetric about $d = \frac{1}{2}$ by construction. This is a statement about the leading-order forms, not the exact ones: the exact Bernoulli $\mathbb{E}[r] = d + (m-1)d(1-d)$ is not itself a function of s (at $m = 1000$ it is 90.01 at $d = 0.1$ and 90.81 at $d = 0.9$, a gap of $1-2d$, and its maximum sits at $d = \frac{1}{2} + 1/(2(m-1))$ rather than at $d = \frac{1}{2}$), while $\mathbb{E}\,\kappa(\mathsf{W})$ *is* exactly a function of s, since $(1-d)^{2w} + d^{2w}$ is symmetric under $d \mapsto 1-d$. Nothing downstream depends on the choice.

By Eq. (S1), the four compressed representations agree to within a constant factor under the null model: S-with-complement costs ms, R costs between $\frac{1}{2}ms$ and $ms+1$, and W costs about $2ms$ in the sparse tail while saturating at m/w. Selection *among* $\{\mathsf{S}, \mathsf{C}\circ\mathsf{S}, \mathsf{R}, \mathsf{W}\}$ therefore buys at most a constant under randomness — randomness gives run containers nothing over sorted arrays. B is the exception and the exception is the entire point: it is a member of the matrix and its expected cost is m/w at every density, so the spread across the full matrix is $(m/w)/(ms) = 1/(ws)$ (Eq. (3)), which is unbounded. At $s = 10^{-6}$ the ratio of the most to the least expensive representation in the matrix exceeds 10^4. What collapses under randomness is the choice among the compressed representations; what does not collapse is the choice between the bitmap and the rest. Correspondingly

it is the *envelope*, not every cell, that is $\Theta(\min(m/w, ms))$ — $\kappa(\mathsf{B})$ is m/w regardless. The crossover of that envelope sits at $ws \approx 1$, i.e. $s^* \approx 1/w$, about 1.6% for $w = 64$ before per-item constants. The U-shape is a genuine theorem of the null model and of the bitmap's flatness. It is not a property of the method, and the method is not in the business of reproducing it — on real data the representations deviate below the null without limit, and that deviation, not the U itself, is the contribution (next).

Proof that density is not a sufficient statistic for cost (the deviation, supporting Eq. (5) and the impossibility argument). Fix m and $d \le \frac{1}{2}$, $k = dm$. Over the sets of density exactly d: $\kappa(\mathsf{B})$ and $\kappa(\mathsf{S})$ are constants, functions of (m,k) alone; $\kappa(\mathsf{R})$ ranges over *every* integer in $[1, \min(k, m-k+1)]$; $\kappa(\mathsf{W})$ ranges from $O(1)$ to $\Theta(\min(k, m/w))$. Two explicit families witness the extremes. $X_{\text{block}} = [0,k)$ has one run, $r = 1$, and its words are one all-one fill flanked by all-zero fills, so $\kappa(\mathsf{W}) = O(1)$. For the spread extreme, take

$$X_{\text{spread}} := \{\, i\lfloor m/k \rfloor \,:\, 0 \le i < k \,\}, \tag{S3}$$

which has $|X_{\text{spread}}| = k$ *exactly* (the largest element is $(k-1)\lfloor m/k \rfloor < m$) — unlike the naive candidate "every $\lceil 1/d \rceil$-th position", which has density $k = dm$ only when $1/d$ is an integer and is not used here for that reason. The stride $\lfloor m/k \rfloor \ge 2$ whenever $k \le m/2$, so every element of X_{spread} is isolated and $r = k$. For $\kappa(\mathsf{W})$: every w-bit word contains at least one element iff the stride is $\le w$, i.e. iff $d \gtrsim 1/w$, in which case every word is dirty and $\kappa(\mathsf{W}) = m/w$; otherwise the k elements fall in distinct words, giving k dirty words separated by $\Theta(k)$ fill groups, $\kappa(\mathsf{W}) = \Theta(k)$. For the intermediate run counts, choose any composition of k into r parts (the run lengths) and any distribution of the $m-k$ zeros into the $r+1$ gaps with the $r-1$ internal gaps non-empty; this is feasible iff $r \le k$ and $r \le m-k+1$, exactly the claimed range (a run needs at least one element, and r runs need at least $r-1$ separating gaps).

The worked case. $m = 10^6$, $A = [0, 5\times10^5)$, $B = [2.5\times10^5, 7.5\times10^5)$. Both operands sit at density exactly $\frac{1}{2}$ — the worst point of Eq. (2), where cardinality alone leaves 500,001 feasible answers and the null model of Eq. (5) predicts $m/4 = 250{,}000$ runs apiece. Both in fact have one run. $\mathsf{B}\times\mathsf{R}$ with a rank index (Eq. (4)) answers the pair in *two rank lookups*; $\mathsf{B}\times\mathsf{B}$ spends 15,625 word operations to reach the same number. Nothing about the density says which situation a selector is in.

The structure ratio. Define $\sigma_\rho(X) := \mathbb{E}_{\text{null}(d)}[\kappa_\rho]/\kappa_\rho(X)$, against the *fixed-cardinality* null throughout ($\mathbb{E}[r] = k(m-k+1)/m$), which conditions on the observable k and is exact; it differs from the Bernoulli form by $O(1)$ and nothing depends on the choice. $\sigma_\rho \equiv 1$ identically for B and S. At every density,

$$\sigma_{\mathsf{R}}(X) \;\ge\; \frac{\max(k, m-k+1)}{m} \;=\; \max\left(d,\; 1-d+\tfrac{1}{m}\right) \;\ge\; \tfrac{1}{2}, \tag{S4}$$

with equality only for the maximally scattered set, and σ_{R} is unbounded above, reaching $\Theta(m)$ on X_{block}. *Proof.* $\sigma_{\mathsf{R}} = \mathbb{E}[r]/r \ge \mathbb{E}[r]/r_{\max}$ with $r_{\max} = \min(k, m-k+1)$ and $\mathbb{E}[r] = k(m-k+1)/m$; since $xy/\min(x,y) = \max(x,y)$, the ratio is $\max(k, m-k+1)/m$, which exceeds $\frac{1}{2}$ because $k + (m-k+1) = m+1 > m$ forces the larger term above $m/2$. This is stronger than a $d \le \frac{1}{2}$-only bound: the restricted form $1-d+1/m$ degenerates to 0.002 at $d = 0.999$, a spurious $500\times$ worst case, where the true bound is $\sigma_{\mathsf{R}} \ge 0.999$. Read as a sentence: real data can be arbitrarily cheaper than random data at the same density, and at most about twice as expensive.

The impossibility (Corollary B), stated at the strength it is actually proved. Any selector whose input is d (or a, b, or any function of them) evaluates one fixed function $f(d)$ on every row of that density; by the range above, $\kappa(\mathsf{R})$ takes every value in $[1,k]$ at $d = k/m$, so $f(d)$ mis-predicts $\kappa(\mathsf{R})$ by an unbounded factor on some rows at every density, and no such selector can distinguish X_{block} from X_{spread}. Two consequences, and the line between what is proved and what is measured must be drawn precisely. (i) A fixed representation cannot exploit σ, since it commits before σ is knowable. (ii) A density-only *cost model* cannot exploit σ either, since it is a function of d and the impossibility applies to it verbatim — but it does *not* follow that such a model is worse than making no selection at all: whether a mis-prediction costs more than a fixed default depends on which cell the wrong prediction picks and that cell's actual cost, neither of which is modelled here. The regret comparison in Results, "Selection must be hoisted, and measured rather than modelled" (Fig. 4b), is a measurement of that question, not a derivation of it. What *is* licensed is narrower and still strong: the proved statement rules out one class of selector — those reading only cardinality — and does not choose among the survivors. A run count or a zone-map occupancy count is $O(1)$ metadata that sees σ perfectly and can perfectly well be fed to a model; probe-and-commit is a different survivor. The theory says only: gate selection on a statistic that sees structure rather than on cardinality. Whether that statistic feeds a structural model or a direct measurement of the candidate kernels is settled by measurement, not derived here.

The null model is a floor, and it is the same floor three times (proofs). Independent placement at fixed density is the *extremal* arrangement for every mechanism used here, always in the same direction. For a representation this is Eq. (S4) above. For a zone map of bin width W_z, the occupied-bin fraction under independence is $p = 1-(1-d)^{W_z}$ against $p = d$ when every run spans a bin; $1-(1-d)^{W_z} \ge d$ for all $d \in [0,1]$ and $W_z \ge 1$ (by convexity of $(1-d)^{W_z}$ in d against its tangent

at $d = 0$), with equality only at the endpoints. Scattering therefore maximises occupancy, so the summary's null cost is an upper bound on its cost and a lower bound on its value. For the prefix filter of the narrowed-contract mode, with prefix length $p(X) = |X| - \lceil t|X| \rceil + 1$, the probability that two independently placed prefixes of length p each share an element, drawn from a universe of size $y^{-1}p^2$, is $1 - e^{-y}$ with $y = p^2/m$: the entire mechanism depends on the operands through that one dimensionless quantity, and any ordering that concentrates rare elements into prefixes makes real prefixes collide less often than scattered ones of the same length. In each case the null is what the mechanism is worth on data with no structure to find; real data deviates from it only in the mechanism's favour; the magnitude of that deviation is not a function of density and so is invisible to any cost model whose inputs are cardinalities; and it is exactly the quantity the mechanism converts into avoided work.

Where the headroom vanishes (Proposition 12, full detail). At $s_A = s_B = \frac{1}{2}$: the cardinality view excludes nothing ($U = m/2$, Eq. (2)); the best list-based cell costs $m/2$ probes (Eq. (3)'s proof) against $\mathsf{B}\times\mathsf{B}$'s m/w word operations, so $\mathsf{B}\times\mathsf{B}$ wins by $w/2$ — a factor of 32 at $w = 64$, before vectorisation; and under the null model $\mathbb{E}[r] = m/4$ and $\mathbb{E}\,\kappa(\mathsf{W}) = (m/w)(1 - 2\cdot 2^{-2w}) \approx m/w$, so neither R nor W offers anything either, and W has degenerated into B with header overhead. So the headroom ratio of Eq. (3) does not merely shrink at $s = \frac{1}{2}$ — it falls below one ($1/(ws) = 1/32$ at $w = 64$), and the correct behaviour of a selector there is to choose $\mathsf{B}\times\mathsf{B}$. (The ratio is of course positive; "negative headroom" is the wrong description for a quantity that has fallen below unity.) This is a statement about the generic case and is false for structured rows at mid density: X_{block} sits at $d = \frac{1}{2}$ with $r = 1$. The honest form is: given only cardinalities, and absent structure, the mid-band is where nothing can be avoided; a structural summary can still find something there, which is why the zone map earns its place. Call this a *metadata floor* or an *identifiability floor* — never an information-theoretic floor, since nothing here lower-bounds the work of an arbitrary algorithm — and name what it floors: what cardinality metadata determines, not what any kernel must spend.

Proof of Eq. (6) and the three regimes (supporting "why the two mechanisms are selected between rather than stacked"). Partition the universe into bins of width W_z and let each position be set independently with probability d. Let K be a bin's cardinality and $p = \Pr[K \geq 1] = 1 - (1-d)^{W_z}$. Then

$$\mathbb{E}[K \mid K \geq 1] \;=\; \frac{W_z d}{p}, \qquad \operatorname{Var}[K \mid K \geq 1] \;=\; \frac{W_z d(1-d)}{p} - \frac{W_z^2 d^2 (1-p)}{p^2}. \tag{S5}$$

Proof. K is zero on the complement of the conditioning event, so $\mathbb{E}[K] = \mathbb{E}[K \mid K \geq 1] \cdot p$ gives the mean; the second moment follows the same way from $\mathbb{E}[K^2] = W_z d(1-d) + W_z^2 d^2$, and the variance is the difference. As $d \to 0$ the conditional distribution collapses onto the single value 1 (coefficient of variation 0.016 at $d = 10^{-6}$, $W_z = 512$), so in the regime where concentration matters most, "an occupied bin" means "a bin with exactly one bit" almost surely: $d/p \to 1/W_z$ as $d \to 0$ and $d/p \to d$ as $d \to 1$. A zone map cannot hand a kernel anything sparser than one bit per bin — that is the whole phenomenon, and it is why the composition is not a product.

Let $\kappa(d) \in (0, 1]$ be the cheapest-representation envelope of Eq. (5), normalised so $\kappa = 1$ is a plain bitmap, and let the zone-map pass cost a fraction c/W_z of a plain-bitmap scan (c counts how many summary streams the pass reads: $c = 1$ for the AND-and-popcount over the occupancy bitmap alone, $c = 2$ if the pass also charges reading both operands' summary streams separately). For A, B drawn independently at matched density d, $p_A = p_B = p$, the composed cost relative to a plain bitmap is Eq. (6), evaluated at the *concentrated* density d/p of Eq. (S5), not at d. Two consequences are exact. The composition never costs more than the zone map alone, since $\kappa \leq 1$ and zone-map-alone is $c/W_z + p_A p_B$. And the composition does not dominate the representation alone: it is floored at c/W_z, whereas $\kappa(d) \to 0$ as $d \to 0$, so below that floor the summary costs more than a sparse representation pays outright. Hence three regimes, not two. In the sparse band $\kappa(d) = wd$, so the composition beats both the representation alone and the bitmap exactly when

$$x e^{-x} \;>\; c/w, \qquad x = W_z d \text{ (expected set bits per bin)}, \tag{S6}$$

a window in bits per bin that is independent of W_z and of the universe. At $c = 1$, $w = 64$ the window is $x \in [0.0159, 5.94]$, i.e. $d \in [3.1 \times 10^{-5}, 1.2 \times 10^{-2}]$ at $W_z = 512$ and $d \in [3.9 \times 10^{-6}, 1.5 \times 10^{-3}]$ at $W_z = 4096$; at $c = 2$ the window is $x \in [0.0323, 5.09]$. The lower boundary has the exact, memorable form $d_{\text{low}} = c/(wW_z)$ (Eq. (7)), the density at which a one-word-per-element representation's own cost wd falls below the zone map's overhead; below it reading the summary costs more than enumerating the operands outright, e.g. $31\times$ their work at $d = 10^{-6}$ ($c = 1, w = 64, W_z = 512$). The upper boundary is where bins stop being empty: at $x \approx 5$ the occupancy $p = 1 - e^{-x}$ exceeds 0.99, the filter removes almost nothing, and its overhead is pure loss. Both boundaries were confirmed against a direct numerical scan of the full envelope (not the sparse approximation) at $W_z \in \{256, 512, 1024, 4096\}$, agreeing with the closed form to three significant figures. A run- or fill-based representation crosses at its own density, wherever its descriptor length falls below m/W_z; that a crossing exists at all is generic. Prefix filtering has the same shape in $y = p^2/m$: for $y \ll 1$ the surviving fraction is y itself, so pruning improves quadratically as prefixes shorten, while for $y > 1$ prefixes collide by default and the index earns nothing.

Refinement is monotone but the zone-map popcount is an upper bound, not an exact count. Partitioning $[0,m)$ into $n_\beta = m/\beta$ bins with per-bin cardinalities a_k, b_k, $\sum_k \max(0, a_k + b_k - \beta) \le |A \cap B| \le \sum_k \min(a_k, b_k)$, the refined interval is contained in the global one and its width is $\sum_k \min(a_k, b_k, \beta - a_k, \beta - b_k) \le U$ — apply Eq. (1) inside each bin and sum. The relation to the zone-map popcount is an inequality, not an equality:

$$\#\{\text{bins with a non-degenerate refined interval}\} \;\le\; \text{popcount}(\text{occ}_A \wedge \text{occ}_B), \tag{S7}$$

with equality iff no bin is saturated on both sides. A bin in which both operands are full is occupied on both sides, so the popcount counts it, yet its refined interval is the single point β — degenerate. The two operationally useful statements survive untouched: a popcount of zero proves the operands disjoint, and the popcount is the number of bins a kernel consulting only occupancy will visit (a kernel that does not know a bin is saturated must still visit it) — but the popcount does not count only the bins whose contribution is genuinely undetermined.

Why the interval width is not a lower bound on work. Nothing in this note lower-bounds the work any algorithm must perform. Choosing a concrete machine model makes the point sharply: in the pure membership-probe model, where an algorithm knows m, a, b in advance and its only primitive is "is position i in A?" or "in B?", with the requirement that it output $|A \cap B|$ exactly, then whenever $a \notin \{0, m\}$ and $b \notin \{0, m\}$ it makes at least $m/2$ probes in the worst case, *independently of s*. *Proof.* Complementing an operand is a bijection on probe sequences (the answer flips) and determines the answer via $|A \cap B| = b - |A^c \cap B|$, so assume $a \le b \le m/2$. The adversary answers "no" to every probe. Let P_A, P_B be the probed sets, q_A, q_B their sizes, $q = q_A + q_B$. If consistency has already been forced ($m - q_A < a$ or $m - q_B < b$) then $q > \min(m - a, m - b) \ge m/2$ and the claim holds. Otherwise choose $A \subseteq \overline{P_A} \cap \overline{P_B}$ with $|A| = a$, possible when $q \le m - a$. The values of $|A \cap B|$ realisable by b-subsets $B \subseteq \overline{P_B}$ span every integer in $[\max(0, b - (m - q_B - a)), \min(a, b)]$, an interval containing two distinct values whenever $q_B < m - b$. So the algorithm cannot have halted while both $q \le m - a$ and $q_B < m - b$, giving $q \ge \min(m - a + 1, m - b) \ge m/2$ since $a, b \le m/2$. This bound is $\Omega(m)$ at *every* density — at $a = b = 1$ it already forces $m - 1$ probes — so it tracks nothing about the interval width. The reason is that a membership-probe model cannot read a sorted array, which is precisely the capability the proof of Eq. (3) assumes. The interval width is therefore not a lower bound in any model considered here, and the nearest model that does give a lower bound gives one insensitive to s; a matching bound in a representation-aware model is a comparison-complexity question with its own literature and is not attempted here.

Worked numerical examples. Universe $m = 10^6$, word width $w = 64$, so $\kappa(\mathsf{B}) = 15{,}625$ words throughout.

A — sparse, $d_A = d_B = 10^{-4}$, $a = b = 100$. Interval $[0, 100]$, width $100 = ms$; 101 feasible answers; $\mathbb{E}|A \cap B| = 0.01$; $\Pr[\varnothing] \approx 99.0\%$; best list cell 100 probes; headroom $1/(ws) = 156\times$; $\mathbb{E}[r]$ under the null ≈ 100, so R buys nothing over S here. The typical answer is 0, and the whole computation should be a disjointness proof.

B — mid-density, unstructured, $d_A = d_B = \frac{1}{2}$. Interval $[0, 500{,}000]$, width $m/2$, 500,001 feasible answers; best list cell 500,000 probes, $32\times$ worse than B×B; $\mathbb{E}[r]$ under the null 250,000, $16\times$ worse; $\mathbb{E}\,\kappa(\mathsf{W})$ under the null $\approx 15{,}625$, W has become B; headroom $1/(ws) = 1/32 < 1$: none.

C — mid-density, structured (the thesis in one line). $A = [0, 5 \times 10^5)$, $B = [2.5 \times 10^5, 7.5 \times 10^5)$; both $d = \frac{1}{2}$, identical to case B by every cardinality statistic. Interval from cardinality unchanged, $[0, 500{,}000]$; $\mathbb{E}[r]$ under the null 250,000; actual $r_A = r_B = 1$; $\sigma_{\mathsf{R}} = 250{,}000$; B×R with a rank index answers in 2 lookups against B×B's 15,625 words. Cases B and C are indistinguishable to any density-based selector and differ in cost by four orders of magnitude; without Eq. (4) this comparison is a run count against a word count and is meaningless.

D — dense, $d_A = d_B = 0.51$. Pigeonhole floor $(2d - 1)m = 2 \times 10^4$, 2% of the universe (factor two, not one), 3.92% of each operand; ceiling 5.1×10^5; width $m(1 - d) = 4.9 \times 10^5 = ms$, consistent with Eq. (2). At $d = 0.99$ the same algebra gives floor $0.98m$, width $0.01m$, and a complement view of 10^4 elements per side — comparable to $\kappa(\mathsf{B}) = 15{,}625$, right at the crossover $s^* \approx 1/w = 1.6\%$. At $d = 0.999$ the complement side is 10^3 and the dense arm of the U is open.

Full derivation of Eq. (8) (the size bound). Since $|A \cap B| \le \min(a, b)$ and $|A \cup B| = a + b - |A \cap B| \ge \max(a, b)$, $J(A, B) = |A \cap B|/|A \cup B| \le \min(a, b)/\max(a, b)$, so $J(A, B) \ge t$ requires $\min(a, b) \ge t \max(a, b)$, and, since $J \le \min(a, b)/b \le 1$ trivially bounds $|A \cap B| \le \min(a, b)$, $|A \cap B| \ge \tau$ requires $\min(a, b) \ge \tau$. Both conditions are necessary and neither is sufficient, so the test only discards pairs and never reports one; survivors still go to an exact kernel. Hypotheses: $a, b > 0$, $t \in (0, 1]$; nothing is assumed about arrangement, about m, or about how either operand is stored. This is the Swamidass–Baldi pruning bound[3] together with the size (length) filter of[16], in the two-sided form standardised by[17] and applied jointly with prefix filtering to Tanimoto over binary vectors by[18]; the contribution of this note is its placement against the mechanisms above, not the bound itself.

Full derivation of Eq. (9) and its two hypotheses. Model a corpus by drawing cardinalities independently with $\log a$ uniform on $[0, \ln K]$ — the density $\Pr(a) \propto 1/a$, i.e. exactly the $1/i$ cardinality spectrum this paper's benchmark protocol mandates, not an approximation to it. Then $\min(a, b)/\max(a, b) \ge t$ is $|\log a - \log b| \le \gamma$ with $\gamma = \ln(1/t)$. For two independent uniforms on

an interval of length $L = \ln K$, the probability that $|X - Y| \le \gamma$ for $X, Y \sim \mathrm{Unif}[0, L]$ is $1 - (1 - \gamma/L)^2$ for $\gamma \le L$ (the complement event, $|X - Y| > \gamma$, has probability $(1 - \gamma/L)^2$ by direct integration over the two corner triangles), giving Eq. (9), with absolute form $(1 - \ln\tau/\ln K)^2$ for the $|A \cap B| \ge \tau$ reading. At $K = 10^4$ the surviving fraction is 0.1449 at $t = \frac{1}{2}$, 0.0479 at $t = 0.8$, 0.0227 at $t = 0.9$ and 0.0111 at $t = 0.95$; at $K = 10^3$, $t = 0.9$ it is 0.0303. Equivalently, examining every pair does 6.9×, 20.9×, 44.0×, 90.0× and 33.0× the work of examining only the survivors. These agree with a 10^5-sample simulation to three decimals at each point. Expanding for small γ gives $2\gamma \int \phi^2$ for any density ϕ of $\log a$, so the governing quantity is the *effective log-spread* $1/\int g^2$ of the size distribution, of which Eq. (9) is the log-uniform case.

Two hypotheses carry real weight. *First*, cardinalities are integers, and $a = b$ passes at every t, so the continuous form is optimistic where sizes are small: drawing integer cardinalities as the generator does leaves 0.0185 rather than 0.0111 surviving at $t = 0.95$, $K = 10^4$. *Second*, Eq. (9) decreases monotonically in K — at $t = 0.9$ the surviving fraction falls from 0.0452 at $K = 10^2$ to 0.0114 at $K = 10^8$ — but that improvement belongs to the log-uniform family, not to heavy tails generally. Under the alternative reading in which *ranked* sizes scale as $1/i$, giving a density $\propto 1/a^2$ whose log is exponential, the surviving fraction is exactly $1 - t$, independent of K: the effective log-spread saturates at two nats however far the sizes are spread, and so does the gain. The favourable scaling in K is therefore claimed for the spectrum this paper benchmarks and not for every skewed corpus.

Full derivation of Eq. (10) (why the size bound does not multiply). Acting at a coarser granularity is not sufficient for savings to compose. The size test decides whether a pair is formed at all, while representation choice and the zone map act inside a pair already formed, so the granularity argument of Eq. (6) predicts a product; the product is not achieved, because both mechanisms read the same statistic. The survivors of Eq. (8) are the pairs with $a \approx b$, and every mechanism below is cheapest exactly when a, b are dissimilar, since its cost tracks $\min(a, b)$ against a fixed $\lceil m/w \rceil$. Under the log-uniform model above, conditioning on $\min / \max \ge t$ raises the mean cost of a survivor by the factor of Eq. (10), whose limit as $t \to 1$ is obtained by Taylor-expanding both the conditional and unconditional expectations of $\min(a, b) = e^{\min(X,Y)}$ in $\gamma = \ln(1/t)$ and taking the ratio; the leading term is $\ln K/2$. Numerically, at $K = 10^4, t = 0.9$ the factor is 4.40 against the limit $\ln K/2 = 4.61$. Simulating the composition directly reproduces the shortfall: the size bound leaves 2.76% of pairs and representation choice costs 1.51% of a bitmap, so a product would predict 0.042% where the composition in fact costs 0.151%, a shortfall of 3.69× once cardinalities are integers; permuting which pairs pass the size test, so the two mechanisms read uncorrelated statistics at the same survival rate, restores the product to within 0.6%, isolating the shared statistic as the cause rather than the granularity. The composition is still strongly worth taking: at these parameters it costs 10.0% of the better of the two mechanisms alone, and a bitmap pass over every pair does 660× its work. What is refuted is only the product; the general rule is that mechanisms compose multiplicatively when they act on different waste *and* are driven by different operand statistics, and sharing a statistic costs the composition Eq. (10)'s factor even across granularities. All values here are analytic, computed over the log-uniform model at $m = 10^6$, and count work rather than time.

Supplementary Note 9: The representation-choice optimisation problem — full proofs

Methods ("Choosing a representation is a different problem from choosing a size" onward) states this development's results as closed forms with their hypotheses and a pointer here. Chunked setting: the universe is partitioned into chunks of M bits, each stored in one representation; $n = M/w$ words to a chunk, e bits to a stored array element. For an operation ω and a representation pair (ρ_A, ρ_B), the cell cost is $C_\omega(\rho_A, \mu_A; \rho_B, \mu_B) = \sum_j c_j N_j$, where $\mu_X = (k_X, r_X, \ell_X + g_X)$ is the operand's metadata, the N_j are counts of primitive items touched, and the $c_j > 0$ are per-item constants entering only through the ratios $\theta_{pw} = c_p/c_w$, $\theta_{mw} = c_m/c_w$, $\theta_{\rho w} = c_\rho/c_w$ for a membership probe, a merge step and a rank lookup respectively, each measured, not derived. Given a corpus, a distribution π over the chunk pairs actually evaluated, and an operation ω, the decision problem is to choose an assignment ρ : chunks $\to \{\mathsf{S}, \mathsf{B}, \mathsf{R}, \mathsf{W}\}$ minimising $C(\rho) = \sum_{(x,y)\sim\pi} C_\omega(\rho(x), \mu_x; \rho(y), \mu_y)$ subject to a byte budget F.

Proof that the storage objective is separable and the query objective is not. $\sum_x \mathrm{bytes}(\rho(x), \mu_x)$ is a sum of per-container terms, so its minimiser is obtained by minimising each term independently. $C(\rho)$ is a quadratic pseudo-Boolean function of the assignment: exhibiting one pair interaction suffices, and the cost model supplies it directly — $C_{\cap\mathrm{card}}(\mathsf{S}, k_A; \mathsf{B}, \cdot) = c_p k_A$ while $C_{\cap\mathrm{card}}(\mathsf{S}, k_A; \mathsf{S}, k_B) = c_m(k_A + k_B)$, so the cost attributable to chunk x depends on $\rho(y)$ for its partner y. This, not the value of any particular threshold, is the structural difference between the two objectives: a storage-optimal assignment is computed one container at a time with no knowledge of the workload; a query-optimal one cannot be, because there is no such thing as the cost of a container, only the cost of a pair.

Proof that a vendor's fixed storage threshold is M/e, and why it is a compile-time constant. Reading three independent container-promotion rules off a released amalgamation of CRoaring (v4.7.2, `third_party/croaring_amalg`) as an instance of the general result: array against bitset promotes iff $ek > M$; array against run promotes iff $\mathrm{bytes}(\mathsf{R}) < \mathrm{bytes}(\mathsf{S})$; bitset against run promotes iff $\mathrm{bytes}(\mathsf{R}) < \mathrm{bytes}(\mathsf{B})$. Each is a two-way comparison of serialized sizes, and the array-to-bitset

threshold $\tau_{\text{stor}} = M/e$ is exactly the cardinality at which the two containers occupy the same bytes. It contains no machine constant c_j, no property of π, ω or F: it is a function of the format alone, which is exactly why it can be a compile-time constant rather than a tuned one. *An aside on non-confluence.* Because each comparison is two-way and which two depends on the container's present type, the same set can end in different representations depending on how it arrived: from an array container the result is a run iff $2+4r < 2k$; from a bitset it is a run iff $2+4r < 8192$. The two disagree on $(k-1)/2 \le r \le 2047$, $k \le 4096$ — for instance $k = 3000, r = 1600$ gives array 6000 bytes, run 6402 bytes, bitset 8192 bytes: arriving as an array the container stays an array (since $6402 \ge 6000$); arriving as a bitset it becomes a run (since $6402 < 8192$), 402 bytes larger than the three-way minimum for the same set. A three-way minimum over all three serialized sizes is confluent and never larger than the present two-way rule.

Proof of Eq. (11) (the crossover as a formula). For a chunk of cardinality k under count-only intersection, promoting from array to bitset is favourable against a bitset partner iff $c_w n < c_p k$, i.e. $k > \tau_\cap = (c_w/c_p)(M/w) = n/\theta_{pw}$ — the promoted side now costs $c_w n$ (a full word scan) against the alternative $c_p k$ (one probe per element). Against an array partner of cardinality k_Y, promotion turns a two-sided merge into a one-sided probe, $c_m(k+k_Y) \to c_p k_Y$, favourable iff $k > (\theta_{pm}-1)k_Y$ with $\theta_{pm} = c_p/c_m$; in particular if $c_p \le c_m$, promotion helps at every k, since a bitset is the cheapest thing to be probed into. Dividing $\tau_{\text{stor}} = M/e$ by $\tau_\cap$ gives Eq. (11)'s ratio $(w/e)\theta_{pw}$; at $w/e = 4$ (CRoaring on a 64-bit host) the storage threshold exceeds the count-only-AND threshold by exactly $4\theta_{pw}$ and no other factor. Substituting the measured symmetric crossover (the cardinality at which an array–array merge stops beating a bitmap–bitmap popcount, $k \approx 64$–128 on the target host) into the three-way relation $\theta_{pw} = \theta_{mw}\theta_{pm}$ reproduces the reported 30–60× divergence between the two objectives from one measurement and no free parameters, under the assumption $\theta_{pm} \approx 2$ (a probe is an index, load, shift and test against a merge step's pointer advance and compare); that assumption is plausible but unmeasured; and measuring the two crossovers together over-determines the model, since θ_{pm} computed two ways must then agree.

Proof of the price-of-a-byte family (why every threshold between $\tau_\cap$ and τ_{stor} is optimal for some price). Introduce a byte price $\lambda \ge 0$ and minimise $C(\rho) + \lambda \cdot \text{bytes}(\rho)$. For the $\{\mathsf{S},\mathsf{B}\}$ decision on a container of cardinality k participating in ν pairs, write $\Delta w(k) \ge 0$ for the per-pair item saving from promotion and $\Delta b(k) = M/8 - ek/8$ for its byte cost. The rule "promote iff $\nu\,\Delta w(k) > \lambda\,\Delta b(k)$" is a threshold rule in k for every fixed λ, ν: $\Delta w(k)/\Delta b(k)$ is non-decreasing in k on $k < \tau_{\text{stor}}$ (numerator non-decreasing, denominator positive and strictly decreasing), so the promoted set is an up-set in k, and for $k \ge \tau_{\text{stor}}$, $\Delta b \le 0 \le \nu\Delta w$ so those k are promoted at every λ. The threshold is non-decreasing in λ, since the promoted set shrinks pointwise as λ grows; it equals $\tau_\cap$ at $\lambda = 0$ (the rule reduces to $\Delta w(k) > 0$) and tends to τ_{stor} as $\lambda \to \infty$ (only $\Delta b \le 0$ survives); and its range is, up to integer rounding, exactly $[\tau_\cap, \tau_{\text{stor}}]$, since the threshold is the crossing point of two continuous functions of k moving continuously between the two endpoints. So $\tau_\cap$ and τ_{stor} are not a right answer and a wrong answer; they are the two limits $\lambda \to 0$ and $\lambda \to \infty$ of one family, and every threshold between them is optimal at some price of a byte. A vendor's fixed threshold is the unique member of that family at which promotion is never paid for in bytes — the largest threshold at which the compute-favourable move is footprint-neutral, since promoting a chunk of cardinality k multiplies its footprint by τ_{stor}/k. One consequence is worth isolating: the threshold depends on ν, the number of pairs a container participates in, so two containers of identical k and different ν have different optima whenever $\lambda > 0$ — under any byte budget the optimal rule is not a function of the container at all, and no per-container rule attains it.

Proof of Eq. (12) and the full statement of the five hypotheses (the fixed-threshold-is-optimal result, stated first in the direction that weakens it). The claim, at its correct strength. If (H1) the candidate set is $\{\mathsf{S},\mathsf{B}\}$ only, (H2) the operation is count-only intersection, (H3) the per-item constants do not depend on resident footprint, (H4) there is no byte budget, and (H5) every chunk faces the same partner mix, then the optimal assignment *is* the fixed threshold $\tau_\cap$ — a machine constant independent of the corpus and of the pairing distribution. *Proof.* Under H1–H2 the cost of a pair is one of the entries derived above; under H5 the promotion decision is the same functional of k for every container; H4 removes coupling through a budget; H3 makes that functional's constants workload-independent. So the unqualified claim "no fixed constant is optimal" is false; each of the five hypotheses must be dropped explicitly, and each drop has an exact witness.

Dropping H1 (admitting R). No rule reading cardinality alone can rank $\{\mathsf{S},\mathsf{B},\mathsf{R}\}$: at fixed k the run count attains every integer in $[1, \min(k, M-k+1)]$, the $\mathsf{R}\times\mathsf{B}$ cost is strictly increasing in it, and the $\mathsf{S}\times\mathsf{B}$ cost does not depend on it, so two containers of the same cardinality can have different optimal representations and any k-only rule mis-ranks one of them — the impossibility argument above, specialised to the container decision.

Dropping H5 (partner mix). Let a container of cardinality k face bitset partners with probability β and array partners of mean cardinality $\bar{k}$ otherwise. Promotion is favourable iff $\beta c_p k + (1-\beta)c_m(k+\bar{k}) > \beta c_w n + (1-\beta)c_p\bar{k}$, giving the threshold of Eq. (12), which interpolates between $\tau^*(1,\cdot) = \tau_\cap$ and $\tau^*(0,\bar{k}) = (\theta_{pm}-1)\bar{k}$: a statistic of the workload, not of the machine, since two corpora with identical containers and different pairing distributions have different optima. *The fixed point.* β is not exogenous, since partners are themselves subject to the same rule; writing $\beta(\tau)$ for the pair-weighted fraction of partners with $k > \tau$, an optimal fixed threshold satisfies $\tau = \tau^*(\beta(\tau), \bar{k}(\tau))$. Both sides are bounded and τ^* is continuous in β, so a

fixed point exists on $[0,M]$ by the intermediate value theorem whenever $\beta(\cdot)$ is continuous; it need not be unique. A library's container threshold is therefore a fixed point of its own workload, which is exactly why it cannot be a compile-time constant without an objective, such as storage, that removes the dependence.

Dropping H4 (byte budget). The threshold is the dual variable of the footprint constraint above and sweeps the whole interval $[\tau_\cap, \tau_{\text{stor}}]$.

Dropping H2 (the operation). For cardinality-only queries the operation dimension collapses and this is not an approximation: union, difference and symmetric-difference cardinalities are recovered from intersection cardinality by inclusion–exclusion, so all four cardinality queries share one cost matrix. But for *materialising* operations the matrix differs in sign: a materialising union with a bitset operand must produce a bitset-sized result, costing n words whatever the other side is, so promotion helps at every k there and only above $\tau_\cap > 0$ for a count — the two thresholds are 0 and $\tau_\cap$, and no single stored assignment is optimal for a workload mixing the two unless one operation dominates.

Dropping H3 (residency), and this is the one the measurements say dominates. c_w is not a constant: a bitset chunk occupies $M/8$ bytes and an array chunk $ek/8$, so promoting a container multiplies its footprint by $M/(ek) = \tau_{\text{stor}}/k$ — $64\times$ at $k = 64$ and $1\times$ at $k = \tau_{\text{stor}}$. τ_{stor} is therefore also the unique threshold at which promotion never inflates the working set; every lower threshold buys items with bytes at an exchange rate of $\tau_{\text{stor}}/k - 1$, and those bytes are paid in c_w, which rises as the corpus leaves cache. This is the only one of the five hypotheses whose failure is not analytic, and it is treated as a measurement question rather than an analytic one, exactly as Eq. (6)'s divergence between a work model and measured time is.

How far a fixed choice can be from optimal. Bounded exactly under H1–H3 and H5, against the family of the previous paragraph: a container at cardinality k paired with bitsets costs $\min(c_p k, c_w n)$ at the optimum and $c_p k$ or $c_w n$ under a fixed τ, and the worst case sits at the far endpoint, $\max_k C(\tau_{\text{stor}};k)/C(\tau_\cap;k) = c_p\tau_{\text{stor}}/(c_w n) = \tau_{\text{stor}}/\tau_\cap = (w/e)\theta_{pw}$, attained at $k = \tau_{\text{stor}}$; symmetrically, running at $\tau_\cap$ inflates the footprint of a $k = \tau_\cap$ container by the same ratio in both directions.

Proof of Proposition 18/19 (metadata settles the decision wherever the decision matters). Two identifiability questions must not be conflated. Eq. (2) says what cardinality determines about the *answer*, pinned only at the density extremes. What it determines about the *decision* is different and more favourable. If $\mu = (k, r, \ell + g)$, every cost above is a closed-form function of (μ_A, μ_B, n, w) and the constants, so the arg min is too: the decision is determinable exactly, in $O(1)$, touching no operand data — the formal content of "selection must be near-free", a consequence of the cost model rather than an engineering aspiration. If $\mu = (k)$ alone, the decision is *not* determinable whenever R is a candidate (the H1-witness above); one extra integer, the run count, restores determinability, and more cardinality precision does not.

Suppose each constant c_j is known only up to a multiplicative factor $\sqrt{\eta}$ in each direction, so any modelled cost is known within a factor $\eta \geq 1$. Order the candidate cells by modelled cost, $C_1 \leq C_2 \leq \ldots$, and define the margin $\Lambda = C_2/C_1$. *Proof.* If $\Lambda > \eta$, even the most adverse admissible perturbation of the constants leaves $C_1 < C_2$, so the true ordering of the two best cells is determined and the selector's choice is correct. If $\Lambda \leq \eta$, then $C_2 \leq \eta C_1$, so the ordering is not determined by μ but the cost of choosing wrong is at most η, by the definition of Λ. So for every pair, either the metadata settles the decision or the decision does not matter to within the precision of the constants, and the undetermined set is an η-neighbourhood of the boundaries derived above rather than a region of unbounded error. This is a different mid-band from the one in Eq. (2): Proposition 2 (main text) says the *answer* is pinned by cardinality only at the density extremes; this result says the *decision* is pinned everywhere except an η-neighbourhood of a boundary, and the two regions have different consequences and must not be merged — at $d = \frac{1}{2}$ cardinality determines nothing about the answer while determining the decision completely (B×B, Supplementary Note 8). The caveat that makes η interesting rather than cosmetic: η is the uncertainty of the *model*, not of the constants alone, and where the model omits the memory system — the same divergence recorded under Eq. (6) — it is not small. That is the derived motivation for measuring a candidate kernel rather than only modelling it, in a narrower and more defensible form than "the theory implies probe-and-commit": it says only that η is large in a named regime, and that a mechanism which reduces η is worth its cost there.

Proof of Eq. (13) (why an advantage survives repairing the incumbent's kernel). Write the total cost of a batch as $C(\rho) = C_{\text{meta}} + \sum_{(A,B)\in\mathscr{P}_{\text{surv}}} \sum_{z\in Z(A,B)} C_{\text{cell}}(\rho(A|z), \rho(B|z))$, where $\mathscr{P}_{\text{surv}} \subseteq \mathscr{P}$ are the pairs not discarded by metadata and $Z(A,B)$ the regions actually visited. $\mathscr{P}_{\text{surv}}$ and $Z(A,B)$ are functions of the metadata and the data only: the size bound reads (a,b), the zone-map test reads bin occupancy, the narrowed-contract mode reads (a,b,t), and none reads a representation — since every representation here is exact and interconvertible, changing ρ changes no membership. Consequently, for a uniform kernel improvement by a factor $\alpha > 1$ (every C_{cell} divided by α), $G(\alpha) = \dfrac{C_{\text{base}}/\alpha}{C_{\text{meta}} + N_{\text{surv}}\bar{E}_{\text{surv}}/\alpha} = \dfrac{C_{\text{base}}}{\alpha C_{\text{meta}} + N_{\text{surv}}\bar{E}_{\text{surv}}}$ by direct substitution, where $C_{\text{base}} = N_{\text{all}}\bar{E}_{\text{all}}$ is the same batch with no avoidance: invariant in α exactly when $C_{\text{meta}} = 0$, and eroding only through the summary's share of the post-avoidance cost. Improving a kernel therefore cannot subsume avoidance, because avoidance changes which work is asked for and a kernel changes only what the asked-for work costs — but the two are not independent, and which savings multiply is settled by the rule of Eq. (10): a zone map reads bin occupancy while

representation choice reads cardinality and run count, so those multiply; the size bound reads cardinality, which representation choice also reads, so those do not, and a container-threshold gain must not be multiplied into a size-bound gain.

The peak, derived. Working inside a chunk both operands occupy (crediting the incumbent its own chunk-level skip in full), against an incumbent already holding the compute-optimal threshold $\tau_\cap$, with the challenger adding a zone map of bin width W_z at overhead c/W_z, the modelled advantage is $G(d) = \kappa_{\mathrm{Ro}}(d)/\big(c/W_z + p_A p_B\,\kappa(d/p)\big)$ with $\kappa_{\mathrm{Ro}}(d) = \min(1, \theta_{pw} w d)$ the two-cell (array/bitset) envelope and $p = 1-(1-d)^{W_z}$ (Supplementary Note 8, Eq. (S5)). In the sparse band, where $d/p \to 1/W_z$ puts a singly-occupied bin above the compute crossover whenever $W_z \le \theta_{pw} w$, $\kappa(d/p) \equiv 1$ and, substituting $p \approx x = W_z d$ for $x \ll 1$, $G(x) = \theta_{pw} w x/\big(c + W_z(1-e^{-x})^2\big)$. Setting $\mathrm{d}G/\mathrm{d}x = 0$ gives $x^* = \sqrt{c/W_z}$ and the peak value of Eq. (13), valid on $(\theta_{pw} w\sqrt{c})^{2/3} \le W_z \le \theta_{pw} w$: below the lower edge d^* would exceed $1/(\theta_{pw} w)$, where κ_{Ro} saturates at 1 and the peak is capped below the formula's value; above the upper edge $\kappa(d/p) < 1$ and the peak saturates at the W_z-independent value $\sqrt{\theta_{pw} w/c}/2$. The same constant sets a natural bin width: since an occupied bin holds at least one bit in W_z positions, a summary hands the kernel nothing sparser than that, and $W_z^* = \theta_{pw} w$ is the width at which a singly-occupied bin sits exactly at the array–bitset compute crossover, linking the zone-map granularity and the container threshold through one measured ratio. At $\theta_{pw} = 16$, $w = 64$, $c = 2$ the valid window is $128 \le W_z \le 1024$ and the saturated ceiling is $11.3\times$; at $W_z = 512$ (the configuration used throughout the figures) the closed form gives $16.0\times$.

Two consequences are stated rather than smoothed. Eq. (13) is a null-model ceiling on structureless data, and by the one-sidedness of σ (Supplementary Note 8), real corpora can only fall below the null, so a measured advantage exceeding the ceiling is evidence of structure and must be reported as such rather than as agreement with the model — a measured range whose lower and middle values sit inside the ceiling and whose top exceeds it is the expected signature of real structure, not a discrepancy to explain away. And Eq. (13) excludes the narrowed contract entirely: Eq. (9) supplies its own factor, and by Proposition 21's rule the two are not multiplied together.

References

1. Bayardo, R. J., Ma, Y. & Srikant, R. Scaling up all pairs similarity search. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*, 131–140, DOI: 10.1145/1242572.1242591 (2007).
2. Xiao, C., Wang, W., Lin, X. & Yu, J. X. Efficient similarity joins for near duplicate detection. In *Proceedings of the 17th International Conference on World Wide Web (WWW)*, 131–140, DOI: 10.1145/1367497.1367516 (2008).
3. Swamidass, S. J. & Baldi, P. Bounds and algorithms for fast exact searches of chemical fingerprints in linear and sublinear time. *J. Chem. Inf. Model.* **47**, 302–317, DOI: 10.1021/ci600358f (2007).
4. Haque, I. S., Pande, V. S. & Walters, W. P. Anatomy of high-performance 2D similarity calculations. *J. Chem. Inf. Model.* **51**, 2345–2351, DOI: 10.1021/ci200235e (2011).
5. Dalke, A. The chemfp project. *J. Cheminformatics* **11**, 76, DOI: 10.1186/s13321-019-0398-8 (2019).
6. Muła, W., Kurz, N. & Lemire, D. Faster population counts using AVX2 instructions. *The Comput. J.* **61**, 111–120, DOI: 10.1093/comjnl/bxx046 (2018). ArXiv:1611.07612.
7. Fu, Y.-X. Statistical properties of segregating sites. *Theor. Popul. Biol.* **48**, 172–197, DOI: 10.1006/tpbi.1995.1025 (1995).
8. Chambi, S., Lemire, D., Kaser, O. & Godin, R. Better bitmap performance with Roaring bitmaps. *Software: Pract. Exp.* **46**, 709–719, DOI: 10.1002/spe.2325 (2016). ArXiv:1402.6407.
9. Lemire, D. *et al.* Roaring bitmaps: Implementation of an optimized software library. *Software: Pract. Exp.* **48**, 867–895, DOI: 10.1002/spe.2560 (2018). ArXiv:1709.07821.
10. Goodwin, B. *et al.* BitFunnel: Revisiting signatures for search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 605–614, DOI: 10.1145/3077136.3080789 (2017).
11. Răducanu, B., Boncz, P. & Żukowski, M. Micro adaptivity in Vectorwise. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 1231–1242, DOI: 10.1145/2463676.2465292 (2013).
12. Klarqvist, M. D. R. Tomahawk: Fast calculation of LD and genotype correlation in large-scale cohorts. https://github.com/mklarqvist/Tomahawk (2017). Software repository, created 2017-07-17.
13. Wu, K., Otoo, E. J. & Shoshani, A. Optimizing bitmap indices with efficient compression. *ACM Transactions on Database Syst.* **31**, 1–38, DOI: 10.1145/1132863.1132864 (2006).
14. Mood, A. M. The distribution theory of runs. *The Annals Math. Stat.* **11**, 367–392, DOI: 10.1214/aoms/1177731825 (1940).

15. Lang, H. *et al.* Tree-encoded bitmaps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 937–967, DOI: 10.1145/3318464.3380588 (2020).

16. Arasu, A., Ganti, V. & Kaushik, R. Efficient exact set-similarity joins. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*, 918–929 (2006).

17. Mann, W., Augsten, N. & Bouros, P. An empirical evaluation of set similarity join techniques. *Proc. VLDB Endow.* **9**, 636–647, DOI: 10.14778/2947618.2947620 (2016).

18. Anastasiu, D. C. & Karypis, G. Efficient identification of Tanimoto nearest neighbors. In *Proceedings of the IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 156–165, DOI: 10.1109/DSAA.2016.23 (2016).

19. Jacobson, G. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, 549–554, DOI: 10.1109/SFCS.1989.63533 (1989).

20. Clark, D. R. *Compact Pat Trees*. Ph.D. thesis, University of Waterloo (1996).

21. Vigna, S. Broadword implementation of rank/select queries. In *Experimental Algorithms (WEA 2008)*, vol. 5038 of *Lecture Notes in Computer Science*, 154–168, DOI: 10.1007/978-3-540-68552-4_12 (Springer, 2008).

22. Morzy, M., Morzy, T., Nanopoulos, A. & Manolopoulos, Y. Hierarchical bitmap index: An efficient and scalable indexing technique for set-valued attributes. In *Advances in Databases and Information Systems (ADBIS)*, vol. 2798 of *Lecture Notes in Computer Science*, 236–252, DOI: 10.1007/978-3-540-39403-7_19 (2003).

23. Lemire, D., Ssi-Yan-Kai, G. & Kaser, O. Consistently faster and smaller compressed bitmaps with Roaring. *Software: Pract. Exp.* **46**, 1547–1569, DOI: 10.1002/spe.2402 (2016). ArXiv:1603.06549.

24. Sandes, E. F. d. O., Teodoro, G. & Melo, A. C. M. A. Bitmap filter: Speeding up exact set similarity joins with bitwise operations. *Inf. Syst.* **88**, 101449, DOI: 10.1016/j.is.2019.101449 (2020).

25. Chaudhuri, S., Ganti, V. & Kaushik, R. A primitive operator for similarity joins in data cleaning. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 5–16, DOI: 10.1109/ICDE.2006.9 (2006).